Scaling Graph Inference by Serving Models as Views

Abstract

Graph machine learning models such as graph neural networks (GNNs) have been routinely queried (by "inference queries") for various graph analytical tasks. Nevertheless, the training and inference process of graph models remain expensive for large graphs. Even when a graph model is provided, the graph data that are needed for answering inference queries may still be unavailable or unpublished, preventing graph inference to scale to other scenarios.

This paper investigates a novel paradigm to scale graph inference by serving graph models as "views". Given a class of local graph models $\mathcal{M}$, a graph model M, and a graph G, we aim to decide if it is possible to "approximate" the output M over G by only referring to a set of local models in $\mathcal{M}$, and their test cases. We introduce a class of memoization structures that package metadata of graph models, test data, and local inference as queryable *graph model views* (GMVs). Based on GMVs, we study three fundamental problems: (1) *Inference*, to approximate the output of M by using GMVs with reduced time cost, and the conditions when this is possible; (2) *Selection*, to decide whether a set of GMVs can be used to reconstruct the output of M, and if so, which to choose; and (3) *Minimization*, to minimize the storage cost of GMVs. For each problem, we establish doability and hardness results, and introduce efficient algorithms for GMV-based inference analysis.

Using benchmark tasks, we experimentally verify that GMVs can significantly reduce the inference cost and scale graph inference to billion-scale graphs. We showcase the applications of GMV in cyber attack detection and bitcoin transaction anomaly detection.

1 Introduction

Graph machine learning (ML) models, such as graph neural networks (GNNs) [43] have been routinely queried to support graph analysis. A graph model M converts a representation of a graph $G = (X, A)$, where X (resp. A) is a matrix of node features (resp. adjacency matrix) of G, and derives a (numerical) matrix Z ("logits") via an inference process. Given a test graph G and a GNN M, an "inference query" $Q(M, v_t, G)$ applies the inference process of M over G to return the desired, user-specific output $Z(v_t)$ (or Z by default). The logits Z can be post-processed to task-specific output for downstream analysis, such as labels for classification, or numerical values for regression analysis. A query workload can be posed to retrieve the output for a set of nodes V_T, as v_t ranges over V_T.

While being desirable, inference queries in large graphs or "big" graph models, remain computationally expensive or even infeasible. For example, the complete graph G, or the model M may not be available due to data incompleteness, privacy or access control [30, 37], or limited API calls. A more likely scenario is that a set of data resources $\{\mathcal{V}_1, \ldots, \mathcal{V}_n\}$, are published from different organizations. Each data source maintains information about historical inference of M over some compounds G_i $i \in [1, n]$ that "resemble" a counterpart over G. A natural interest is to make the best use of such observations and their results as "views", to *approximately* answer the inference query as fast, and accurate as possible.

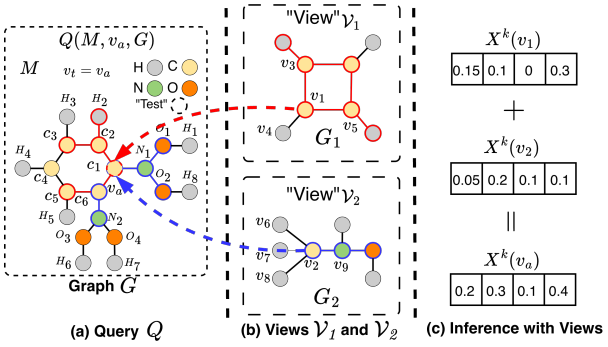


Figure 1: GNNs-based Molecular Property Prediction referring to "views" $\mathcal{V}_1$ and $\mathcal{V}_2$ only to answer inference query Q.

Example 1: Fig. 1 illustrates a molecular graph G of the compound $C_6H_8N_2O_4$, which contains a benzene ring with two $N(OH)_2$ substituents. Nodes represent atoms and edges represent chemical bonds. To validate whether a compound has a desired property in drug designing, a 2-layers vanilla GNN [40] M with node update function $X_v^k = \sigma(\Theta \cdot \text{AGG}(X_u^{k-1}, \forall u \in \mathcal{N}(v)))$ can be trained with atom-level (X) and edge-level features (A) derived from G, and outputs four molecular properties: toxicity, solubility, polar surface area, and polarizability, indicated by numeric probabilities normalized in $[0, 1]$. A chemist designates a targeted atom of interest v_a in G, and issues an inference query $Q(M, v_a, G)$. Such a query can be evaluated via the inference process of M. Nevertheless, the chemist may not have the entire input G or the model M, or does not have the resources to retrain M or perform the inference over large G, yet still expect a fast, estimated answer.

Not all is lost. There are two molecular graphs G_1 and G_2 representing analysis over Cyclobutadiene (C_4H_4) and methylhydroxylamine (CH_5NO) published by other medical centers at *e.g.,* Hugging Face [3], along with output results. *Can we answer Q by consulting the results from G_1 and G_2?* An analogy is to consider this as a familiar scenario of *view-based query processing*, treating both as graph "views". While G_1 and G_2 are not strictly subgraphs of G, they exhibit functional and topological similarities that may be "indistinguishable" by the inference computation within G.

Without the need to directly perform inference of M over G from scratch, we may derive a *matching* from G_1 and G_2 to their counterparts that participated inference process in G, such that the latter can be "simulated" by an inference over G_1 and G_2, with output approximately restored via "scalars" derived from metadata (*e.g.,* degrees, edge weights). The query Q over G thus can be evaluated by only referring to G_1 and G_2, with two cheap calls of M inference.

How could this be made possible? Observing the "matching" of v_1 and v_2 to v_a, we can *represent* the k-th layer embedding of v_a as:

$$
\begin{aligned}
X_{v_a}^k(\text{in } G) &= \sigma\Big(\sum_{u \in \mathcal{N}(v_a)} \Theta^k X_u^{k-1}\Big) = \sigma(\Theta^k \cdot (X_{c_2}^{k-1} + X_{c_6}^{k-1} + X_{N_1}^{k-1})) \\
&= \sigma(\Theta^k \cdot (X_{v_3}^{k-1} + X_{v_5}^{k-1})) + \sigma(\Theta^k \cdot (X_{v_9}^{k-1})) \\
&= X_{v_1}^k(\text{in } G_1) + X_{v_2}^k(\text{in } G_2)
\end{aligned}
$$

where X_v^{k-1} (resp. X_v^k) is the embedding of a node v at the $(k-1)$-th (resp. k-th) layer; σ is an activation function, $N(v)$ refers to the neighbors of node v, and Θ refers to the learned weight matrix (same across all layers in the GNN).

That is, we can reconstruct the embedding $X_{v_a}^k$ by "rescaling" the inference output from G_1 and G_2 according to the degree information. Indeed, the inference for $X_{v_a}^k$ has a fraction contributed from a benzene ring structure that can be "simulated" by a same process for v_1 in G_1; similarly, the other fraction contributed by the $N(OH)_2$ substituents can be recovered by directly rescaling the counterpart for v_2 in G_2. Such neighborhood metadata (*e.g.,* degrees, edge weights/attentions, or hyper-parameters) can be retrieved and reused, in constant time via memoization structures.

The above example indicates the possibility of directly assemble inference results from a set of "views" that resemble original inference results over similar graph data sources. Better still, we expect a bounded inference cost with a time that is only relevant to the size of views, regardless of how big G and M is. □

Analogizing view-based query processing [13], we call this novel strategy *view-based graph inference*. Although desirable, several technical challenges remain.

- How to decide when an inference query Q can be evaluated by such "views" without referring to the original graph G?
- How to reduce the size of views that are needed for such evaluation, hence the computational cost?
- How to evaluate the inference query by only accessing the inference views with reduced inference cost?

Contribution. We make the following contributions.

(1) We introduce *graph model views* (GMVs) $\mathcal{V}$, a class of memorization structures for view-based graph inference. Likening "views" as queries and their results, a GMV $\mathcal{V}$ packages a "memory" graph $G_{\mathcal{V}}$, an auxiliary structure $\tau_{\mathcal{V}}$ for useful metadata, and the inference output of a graph model over $G_{\mathcal{V}}$. The view-based inference aims to refer GMVs $\mathcal{V}$ only to (approximately) answer an inference query $Q(M, v_t, G)$.

(2) We study three fundamental problems for view-based graph inference. These analysis provide theoretical justification and algorithm components for view-based inference algorithms.

(a) *View selection.* Given $Q(M, v_t, G)$ and a set of GMVs $\mathcal{V}$, decide whether it suffices to refer to $\mathcal{V}$ alone to answer Q, and identify a minimal subset of $\mathcal{V}'$ that can best serve the need. We provide an efficient selection algorithm with optimality guarantees, quantified by a coverage property that measures how well their matches in G cover the indistinguishable fraction between the memory graphs and the original graph G.

(b) *View minimization.* Given GMVs $\mathcal{V}$, we introduce preprocessing algorithms to obtain a minimal representation $\mathcal{V}_m$ of $\mathcal{V}$ that preserves $Q(M, v_t, G)$ as much as possible. We present two strategies. The first compresses memory graphs into a canonical form by finding nodes that are equivalent in terms of inference process; the second further prunes nodes with contribution dominated by others in terms of inference and evaluation of Q.

(c) *Inference using views.* We introduce efficient algorithms that refers to GMVs $\mathcal{V}$ to approximately answer Q, and provide a sufficient condition when an exact solution is doable. The algorithm scales the result from selected, minimized GMVs. To best reuse GMVs and reduce memory and inference overhead, we introduce a forgetting curve model to keep useful GMVs as long as possible.

(3) We experimentally verify the effectiveness and efficiency of the proposed GMV framework. Our results demonstrate that with *graph-model-as-a-view*, our methods can substantially reduce the inference cost of representative GNNs such as GCNs, GAT and GraphSAGE by 75.21%-90.53% on average across the studied large-scale graphs, with little to no loss (at most 1.18% and an average of 0.37% lose across all tested large-scale datasets) in inference accuracy. We further showcase the real-world practicality of GMV through case studies in real-world scenarios, including cyber attack detection and anomaly detection from bitcoin transaction.

Related Work. We categorize related work as follows.

View-based Graph Querying. Several methods have been developed to leverage views for graph pattern query evaluation [13, 29, 42, 52]. [13] proposed an efficient and effective view-based query evaluation for pattern queries. It characterizes what pattern queries can be answered using views by pattern containment. [29] extended the use of views from exact query answering to query approximation and studied subgraph isomorphism query answering using views. It introduces the lower and upper approximation of graph queries using views by considering both simulation and subgraph patterns, based on both graph simulation and subgraph isomorphism. More recently, [42] applied view-based techniques to query evaluations in knowledge graphs. The algorithms in [42] accelerate knowledge discovery process by (1) materializing views as base graphs for summaries to support summarized knowledge search; and (2) developing a practical summarization algorithm as view discover process. [52] developed an automatic view selection in graph databases by proposing an *extended graph view*. The algorithm in [52] ensures that the selected view contains all the edge-induced subgraphs to answer the subgraph and supergraph queries simultaneously.

These prior works focus only on graph query evaluation and do not address the challenge of large-scale graph inference for graph machine learning models. By contrast, (1) we take the first step toward view-based methods to scale large-scale graph inference queries; (2) We propose a "graph model views structure GMVs to encapsulate graph models, test graphs and necessary metadata; and (3) we design efficient view-based graph inference at scale with a time cost that is independent with graph size.

Accelerating Graph Inference. Several approaches have been developed to accelerate GNN inference in large graphs [14, 28, 32, 53]. NeuralSparse [53] adopts a learning-based approach that trains supervised DNNs to prune task-irrelevant edges. It leverages both structural and nonstructural features to guide the sparsification processes. AdaptiveGCN [28]learns an edge predictor to determine and remove task-irrelevant edges, thereby accelerating the inference process of on CPU/GPU clusters. However, its design is tightly coupled with GCN architecture and lacks model-agnostic applicability. Dspar [32] develops a model agnostic approach, by removing edges with similar "importance" estimated by circuit-inspired

resistance measures. [14] introduces a model-agnostic, inference-friendly graph compression scheme that preserve GNNs inference. These methods do not address how to exploit and serve external data resources to scale graph inference with quality guarantees. In this work, we integrate data compression and graph query using views to answer inference queries, marrying the advantages of both tracks. This has not been addressed by prior approaches.

Federated and Distributed Graph Computation. Federal learning methods such as FGSSL [25] address structure heterogeneity and the non-IID data challenges in local models and explore similarity in data distributions to distill knowledge from a global model into local models. [7, 41] surveyed recent advancements in distributed and parallel graph models. Distributed graph models train GNNs in a distributed cluster by partitioning the workload through data parallelism [5, 9, 54], model parallelism [31, 45, 55], or hybrid strategies [15, 46]. These methods aim to achieve computational efficiency via data or model parallelization. However, they fundamentally differ from our work in two aspects: (1) inference using views focuses on scaling graph inference by best exploiting external graph data and precomputed inference results, rather than training or inference to generalize to a global model; (2) GMVs allow memory graphs that are *not* necessarily partitions or subgraphs of the original graph. On the other hand, our algorithms are partition friendly, and readily benefit from effective graph parallelization strategies (see Appendix).

2 Graphs and Graph Models

Graphs. A graph $G = (V, E)$ has a set of nodes V and a set of edges $E \subseteq V \times V$. Each node v carries a tuple $T(v)$ of attributes and their values. The size of G, denoted as $|G|$, refers to the total number of its edges, *i.e.,* $|G| = |E|$. Given a node v in G, the *L-hop neighbors* of v, denoted as $N^L(v)$, refers to the set of all the nodes in the L-hop of v in G. The *L-hop subgraph* of a set of nodes $V_s \subseteq V$, denoted as $G^L(V_s)$, is the subgraph induced by the node set $\bigcup_{v_s \in V_s} N^L(v_s)$.

Graph Models. A graph (ML) model $\mathcal{M}$ is a function that takes as input a featurized representation $G = (X, A)$ to an output embedding matrix Z, *i.e.,* $\mathcal{M}(G) = Z$. Here X is a matrix of node features, and A is a (normalized) adjacency matrix of G. [1] In this work, we consider graph neural networks (GNNs) and show that our methods are generally applicable to mainstream GNNs and their specifications.

Inference. We follow a query language perspective [6, 18] to characterize the inference process of a graph model. An inference process refers to the computation of a composition of *node update functions*. Given a GNN $\mathcal{M}$ with L layers, a node update function M_v uniformly computes the embedding of each node v at each layer k ($k \in [1, L]$), with a general recursive formula as

$$x_v^k = M_v^k(\Theta^k, \mathsf{AGG}(X_u^{k-1}, x_v^{k-1}, \forall u \in N^k(v)))$$

which is specified by (1) the learned model parameters Θ^k, (2) an aggregation function AGG (*e.g.,* $\sum$, CONCAT), and (3) the neighbors of v that participate in the inference computation at the k-th layer (denoted as $N^k(v)$). When k=1, X_v^0=$X_v \in X$, *i.e.,* the input features.

[1] A feature vector X_v of a node v can be a word embedding or one-hot encoding [17] of $T(v)$. A is often normalized as $\hat{A} = A + I$, where I is the identity matrix.

The *inference process* of a GNN $\mathcal{M}$ with L layers takes as input a graph $G = (X, A)$, and computes the embedding x_v^k for each node $v \in V$ at each layer $k \in [1, L]$, by recursively applying the node update function. A GNN $\mathcal{M}$ has a *fixed* inference process, if its node update function is specified by fixed input model parameters, layer number, and aggregator. It has a *deterministic* inference process, if $M(\cdot)$ always generates the same embedding for the same input.

We consider GNNs with fixed, deterministic inference processes. In practice, such GNNs are desired for consistent and robust performance. For simplicity, we assume that M_v specifies a proper set of neighbors that participate the inference process as $N(v) \subseteq \{u | (u, v) \in E \text{ or } (v, u) \in E\}$. This allows us to include GNNs that exploits neighborhood sampling (such as GraphSAGE), and directed message passing into discussion. In general, inferences of representative GNNs are in PTIME [11, 56].

GNN Classes. A set of fixed, deterministic GNNs $\mathbb{M}$ belongs to a *class* of GNNs $\mathbb{M}^L$, if for every GNN $\mathcal{M} \in \mathbb{M}$, (1) $\mathcal{M}$ has L layers, and (2) $\mathcal{M}$ uses the same form of node update function M_v^k, for each node $v \in V$ and $k \in [1, L]$. Table 6 (see Appendix) enlists mainstream GNNs with node update functions. Other GNNs are mostly their variants that differs in the choices of aggregation function and other polynomial-time computable operators.

Inference Query. An *inference query* Q is a triple $Q = (M, v_t, G)$ where G is the input test graph, M is a fixed, deterministic GNN model from a class of GNNs $\mathcal{M}^L$ with L layers, and v_t (can be $\emptyset$) is a designated target node. In practice, to answer Q, the model M may provide an API to be invoked, which requiers as input $G = (X, A)$ and perform an inference process to return a task-specific result based on the output-layer embedding $Z(v_t) = \mathbf{x}_{v_t}^{(L)}$.

We define the *result* of Q as (a) the embedding matrix Z, if v_t is not specified *i.e., Q*=$(M, \emptyset, G)$; or (b) $Z(N^L(v_t)) = \bigcup_{v \in N^L(v_t)} Z(v)$, *i.e.,* a set of embeddings that includes all participating nodes in the inference computation of v_t, which involves up to $N^L(v_t)$ in G, for any GNN $M \in \mathcal{M}^L$ with L layers. The result Z is often referred to as the *logits vectors*, and can be further processed (e.g., via softmax) to produce prediction scores or labels. As the inference process is fixed and deterministic, the result of Q is determined by G and M.

3 View-based Graph Inference

To enable view-based inference analysis, we introduce a view structure, notably, *graph model views* (GMVs). Graph model views are a set of *memoization* structures that record information of a historical processing of an inference query $Q(M, v_t, G)$.

3.1 Graph Model Views: A Characterization

Given a GNN M, a *graph model view* (GMV) is a triple $\mathcal{V} = (G_{\mathcal{V}}, \mathcal{T}_{\mathcal{V}}, Z_{\mathcal{V}})$, which contains (a) a *memory graph* $G_{\mathcal{V}}$, a graph over which an inference of M is performed, by an inference query $Q(M, v_t, G_{\mathcal{V}})$; (b) a memoization structure $\mathcal{T}_{\mathcal{V}}$, to store the auxiliary metadata (to be discussed); and (c) $Z_{\mathcal{V}}$, the result of Q.

In an analogy of database views, we say a GMV $\mathcal{V}$ is a *virtual view* if it only contains a memory graph, *i.e.,* $\mathcal{V} = (G_{\mathcal{V}}, \emptyset, \emptyset)$, while the structure $\mathcal{T}_i$ and output Z_i are to be derived as needed; Otherwise, it is *extended*, with pre-computed $\mathcal{T}_i$ and Z_i to serve.

Algorithm 1 : GVInf

Input: An inference query $Q(M, v_t, G)$, a GNN model $M \in \mathcal{M}^L$, relation R^L, a GMV $\mathcal{V}$;
Output: The result of Q.

```
set Z := ∅;
/* extension (once-for-all) */
if V is virtual then
    set T_V := ∅, set Z_V := ∅;
    Construct T_V := {(v, R^L(v)), S_e} from R^L;
    Compute Z_V by local inference;
/* reconstruction */
for v ∈ N^L(v_t) do
    Retrieve R^L(v) from T_V;
    Initialize X_v := Z(v'), where v' ~ Uniform(R^L(v));
    Retrieve S_u from T_V, u ∈ N(v);
    Rewrite M'_v with M_v and S_u;
Restore Z with recursive computation using M'_v;
return Z;
```

Figure 2: Procedure GVInf**: inference with a single view.**

Remarks. GMVs are consistently characterized by the semantics of inference queries Q. Two different GMVs may have a same memory graph but different test nodes, involve a same test node v_t with different memory graphs, or agree on v_t and memory graph, yet bookkeep different results from distinct GNNs from a class $\mathcal{M}^L$.

View-based Graph Inference. The task of *view-based graph inference* has a general form below (as illustrated in Fig. 2):

- **Input**: A graph G, a GNN $M \in \mathcal{M}$, an inference query $Q(M, v_t, G)$, and a set of GMVs $\mathcal{V}$.
- **Output**: an (approximate) result of Q, computed by referring to $\mathcal{V}$, rather than an inference over G from scratch.

Specifically, (1) if $\mathcal{V}$ are virtual GMVs, one invokes at most $|\mathcal{V}|$ local inference queries over the memory graphs $\mathcal{G}_{\mathcal{V}} = \bigcup \mathcal{G}_{\mathcal{V}_i}$; or (2) Otherwise, assemble the extended GMVs (with Z_i, $\mathcal{G}_{\mathcal{V}_i}$ and $\mathcal{T}_{\mathcal{V}_i}$ from each GMV $\mathcal{G}_{\mathcal{V}_i}$ to reconstruct the result for Q.

Ideally, the result of Q can be exactly reconstructed by *only* referring to $\mathcal{V}$ alone, without visiting G. This is desirable as (1) the overall inference cost is determined by the size of GMVs rather than $|G|$; (2) view-based inference is learning-free and post-hoc, which only require data owners to post GMVs without revealing details of M, raw attribute values, or complete G; (3) the model agnostic algorithms are applicable to any GNN $M \in \mathcal{M}^L$ as long as they adopt a same form of node update function.

3.2 View-based Inference: Doability

We next investigate when the view-based graph inference is doable. To this end, we provide a *sufficient* condition that can be checked in PTIME, in terms of node bisimulation.

General Idea. Bisimulation [1] is a fundamental notion that establishes when two nodes (states) in a labeled graph (transition system) cannot be distinguished by an external observer, and has been exploited as data compression and minimization strategies for graph computation [14]. Given a test node $v_t \in G$, our idea considers graph inference computation for v_t over G as the "observer", and identify the node pairs between G and $G_{\mathcal{V}}$ that cannot be distinguished by the inference process, such that it can be computationally simulated by the inference over $G_{\mathcal{V}}$ directly, leveraging properly stored neighborhood statistics.

We start by introducing a notation of *L-bisimulation*. Given two graphs $G_{\mathcal{V}}$ and G, a test node v_t in G, and an integer L, let $G^L(v_t)$ be the L-hop subgraph of v_t in G. An L-bisimulation relation R^L *at* v_t is a binary relation between the nodes of $G_{\mathcal{V}}$ and G, such that

- (1) there exists a node v'_t in $G_{\mathcal{V}}$ where $X^0_{v_t} = X^0_{v'_t}$; and
- (2) for each pair $(v, v') \in R^L$, (a) $X^0_v = X^0_{v'}$, and (b) for *every edge* (u, v) in $G^L(v_t)$, there is a *match* (u', v') in $G^L_{\mathcal{V}}(v'_t)$, such that $(u, u') \in R^L$; and for *every edge* (u', v') in $G^L_{\mathcal{V}}(v'_t)$, there is a match (u, v) in $G^L(v_t)$, such that $(u, u') \in R^L$.

We say v'_t and v_t are *L-bisimilar* if an L-bisimulation between $N^L(v'_t)$ and $N^L(v_t)$ contains (v_t, v'_t).

We can verify the following properties of L-simulation.

Lemma 1: *Given G with v_t, $G_{\mathcal{V}}$, and integer L, (1) an L-bisimulation relation R^L is an equivalent relation; and (2) a maximum L-simulation R^L can be computed in $O(|G_{\mathcal{V}}| + |N^L(v'_t)||G^L(v_t)|)$ time.* □

One can verify that L-bisimulation is reflexive, symmetric, and transitive. The maximum L-simulation can be computed by a verification procedure (denoted as L-Bisim, not shown) that (1) verifies the existence of v'_t in $O(|G_{\mathcal{V}}|)$ time, and (2) computes the maximum bisimulation relation between $N^L(v'_t)$ and $N^L(v_t)$. This can be implemented by invoking fast bisimulation algorithms [19, 33], in $O|N^L(v'_t)||G^L(v_t)|)$ time. Here L is in practice small for GNNs.

Theorem 2: ***(Doability of View-based Inference)***. *Given a* GNN *class $\mathcal{M}^L$, for any* GNN *$M \in \mathcal{M}^L$ with node update function M_v, an inference query $Q(M, v_t, G)$ can be answered by only referring to a* GMV *$\mathcal{V}$, if there exists a node v'_t in $G_{\mathcal{V}}$ that is L-bisimilar to v_t.* □

We next provide a constructive proof of Theorem 2. Given an inference query $Q(M, v_t, G)$ with M from a GNN class $\mathcal{M}^L$, and a GMV $\mathcal{V}$, assume there is a node v'_t in $G_{\mathcal{V}}$ that is L-bisimilar with v_t (verified by procedure L-Bisim). We clarify (a) the construction of $\mathcal{T}_{\mathcal{V}}$ for GMV $\mathcal{V}$, if $\mathcal{V}$ is virtual; and (b) provides a simple procedure, denoted as GVInf, to compute $Q(M, v_t, G)$ by referring to the extended GMV $\mathcal{V}$ only. This procedure is used as a "building block" for our general view-based inference algorithms, when the sufficient condition may not hold for a single GMV (see Section 4).

View-based Graph Inference. We outline the procedure GVInf. It takes as input an inference query $Q(M, v_t, G)$, $N^L(v_t)$, $N^L(v'_t)$, and the maximum L-bisimulation R^L computed by procedure L-Bisim, and evaluates Q with two steps below.

(1) **Extension**. If $\mathcal{V}$ is virtual, GVInf performs a "once-for-all" computation to extend $\mathcal{V}$ by constructing $\mathcal{T}_{\mathcal{V}}$ and $Z_{\mathcal{V}}$. (a) Given the node update function M_v, $\mathcal{T}$ is a set of key-value pairs that compactly encodes R^L with the bisimilar node pairs $(v, R^L(v))$, and for each edge e in $N^L(v_t)$, the number of its matches S_e. (b) It then performs a local inference of M over $N^L(v'_t)$ to store $Z_{\mathcal{V}}$.

(2) **Reconstruction**. Given the GNN class $\mathcal{M}^L$ with update function M_v, it *represents* M_v in terms of a counterpart that directly

GNNs	Node Update Function M_v (at G)	equivalent rewriting w. $M_{v'}$; scaling factors are marked in red	notes
Vanilla [40]	$X_v^k = \sigma(\Theta \cdot \text{AGG}(X_u^{k-1}, \forall u \in \mathcal{N}(v)))$	$X_v^k = \sigma(\Theta \cdot \text{AGG}(\mathbf{s_{e(v',u')}} X_{u'}^{k-1}, \forall u' \in \mathcal{N}(v'))$	AGG: $\sum$ or AVG; for AVG, need to multiple by RF_v
GCN [27]	$X_v^k = \sigma(\Theta^k(\sum_{u\in\mathcal{N}(v)} \frac{1}{\sqrt{deg_u deg_v}} x_u^{k-1}))$	$X_v^k = \sigma(\Theta^k(\sum_{u'\in\mathcal{N}(v')} \frac{1}{\sqrt{\text{deg}_v}} \mathbf{s_{e(v',u')}} x_{u'}^{k-1}))$	deg_v: degree of node v in G, topology sensitive
GAT [44]	$X_v^k = \sigma(\sum_{u\in\mathcal{N}(v)} \alpha_{uv}\Theta^k X_u^{k-1}))$	$X_v^k = \sigma(\sum_{u'\in\mathcal{N}(v')} \mathbf{s_{e(v',u')}} \Theta^k X_{u'}^{k-1}))$	weight sensitive
GraphSAGE [20]	$X_v^k = \sigma(\Theta^k \cdot (X_v^{k-1} \|\| \text{AGG}(X_u^{k-1}, \forall u \in \mathcal{N}(v))))$	$X_v^k = \sigma(\Theta^k \cdot (X_{R^L(v)}^{k-1} \|\| \text{AGG}(\mathbf{RF_v} \times \mathbf{s_{e(v',u')}} X_{u'}^{k-1}, \forall u' \in \mathcal{N}(v'))))$	$\|\|$: concatenation; AGG: AVG
GIN [49]	$X_v^k = \sigma(\text{MLP}((1+\gamma)x_v^{k-1} + \sum_{u\in\mathcal{N}(v)} x_u^{k-1}))$	$X_v^k = \sigma(\text{MLP}((1+\gamma)x_{v'}^{k-1} + \sum_{u'\in\mathcal{N}(R^L(v))} \mathbf{s_{e(v',u')}} x_{u'}^{k-1}))$	

Table 1: Rewriting of node update functions for mainstream GNN classes (scaling factors highlighted in red, node v' is randomly chosen from $R^L(v)$).

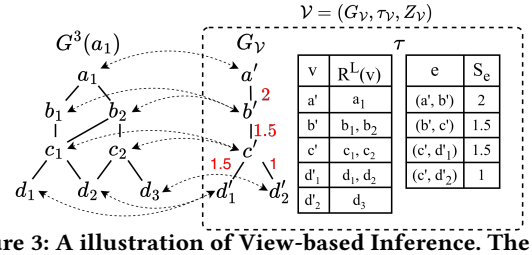


Figure 3: A illustration of View-based Inference. The inference for test node a_1 in $G^3(a_1)$ is "simulated" by a counterpart for a' that 3-bisimilar to a_1 in a GMV, for GNN class $\mathcal{M}^3$.

operates on $R^L(v)$ in $G_\mathcal{V}$. (a) For each node v in $N^L(v_t)$, it retrieves $R^L(v)$ from $\mathcal{T}$, which is the set of its L-bisimilar counterparts (an equivalent class), and initializes X_v with $Z(v')$ where v' is a randomly selected node in $R^L(v)$. (b) It then replaces, for each X_u^{k-1} ($u \in N(v)$) in M_v, a *scaling factor* s_u at query-time, where s_u is determined by the aggregators of M_v (*e.g.*, $\sum$, AVG), the neighborhood statistics of u (such as node degrees, edge attentions, etc), and $S(u, v)$, a scaling factor to be used to scale the "message" from u to v towards the test node; all bookkept in $\mathcal{T}_\mathcal{V}$ from $\mathcal{V}$. Such scaling factors can be pre-determined for main-stream GNN classes, or directly included in $\mathcal{T}$ during the once-for-all extension stage. Table 1 summarizes the scaling factors for mainstream GNNs.

Example 2: Consider an inference query $Q(M, a_1, G)$ for a Vanilla GNN $M \in \mathcal{M}^3$ (L=3), and the 3-hop neighbor graph $G^3(a_1)$ as shown in Fig. 3. A GMV $\mathcal{V}$ is initialized with a memory graph $G_\mathcal{V}$. The nodes with same labels a, b, c, d have the same input features except $X_{d_1}^0 = X_{d_2}^0 \neq X_{d'_2}^0$.

(1) In the extension phase, procedure GVInf invokes procedure L-Bisim to verify if a 3-bisimilar exists, and derives the following R^3: $\{(a', a_1), (b', b_1), (b', b_2), (c', c_1), (c', c_2), (d'_1, d_1), (d'_1, d_2), (d'_2, d_3)\}$; and (2) for each edge e in $G^L(a_1)$, at least one match in its match set. It then performs local inference of M over $G_\mathcal{V}$ to store $Z_\mathcal{V}$.

(2) In the reconstruction phase, GVInf reconstructs the inference over the view graph G_V using $\mathcal{T}_\mathcal{V}$ and $Z_\mathcal{V}$. For each node $v \in G_3(a_1)$, it first initializes its feature representation as $X_v := Z(v')$ where node v' is randomly chosen from $R_3(v)$. It then multiplies each neighbor message X_u^{k-1} in M_v by the corresponding scaling factors retrieved from T_V. Since M is a 3-layer Vanilla GNN, the scaling factors for all edges are derived accordingly and bookkept in $\mathcal{T}_\mathcal{V}$. For example, consider edge (b', c'), there are three matches in $G^3(a_1)$ for for edge (b', c') and $|R^3(b')| = 2$. Hence, the scaling factor associated with (b', c') is $s_{e(b',c')} = \frac{3}{2}$. All such scaling factors highlighted in red in Fig. 3 are precomputed and stored in $\mathcal{T}_\mathcal{V}$, enabling once-for-all reuse. Finally, GVInf recursively applies the updated node function M'_v as illustrated in Table 1 across G_V to reconstruct Z as the inference result of Q. □

Correctness and Cost. We show that GVInf correctly reconstruct $Q(M, v_t, G)$ from $Q(M, R^L(v_t), G_\mathcal{V})$ for a GNN $M \in \mathcal{M}^L$ with an inductive analysis on the inference process of M. The inference executes, at each layer and each node v in $N^L(v_t)$, a node update function to gather the embeddings of all neighbors of v, invoke M_v and update X_v, and sends out the updated embedding to v's neighbors. As there always exists a L-bisimilar node v', such behavior can be correctly simulated by a same process that invokes $M_{v'}$. The only difference is that due to different neighbors, the constant factor that scales X_v differs from X'_v. Such value difference can be eliminated by "re-scale" X'_v with the scaling factor, derived from the recorded edge matches in $\mathcal{T}$. As this computation is consistently simulated at each node $R(v)$, the entire inference process for v_t, which involves only the nodes in $N^L(v)$ in an L-layer GNN M, can be correctly reconstructed by at most L rounds of inference, treated as a composite function that recursively apply M_v up to L times.

For the time cost, observe that GVInf incurs a *once-for-all* overhead in $O(|\mathcal{V}||N^L(v'_t)|^2)$ time to extend all virtual GMVs, with at most $|\mathcal{V}|$ function calls of the inference of model M. The view-based inference is a direct scaling of recorded embeddings, in total $O(L|N^L(v'_t)|)$ time. As L is typically small ($L \leq 4$), it brings down the inference cost of $Q(M, v_t, G)$ from quadratic-time full-graph cost to a bounded, linear time determined by L and $|G_\mathcal{V}|$.

This completes the proof of Theorem 2.

The above results can be readily extended to a set of GMVs $\mathcal{V} = \{\mathcal{V}_1, \ldots \mathcal{V}_n\}$. Define the memory graph $\mathcal{G}_\mathcal{V}$ of $\mathcal{V}$ as the union of the memory graphs $G_{\mathcal{V}_i}$ of $\mathcal{V}_i \in \mathcal{V}$.

Corollary 3: *An inference query $Q = (M, v_t, G)$ ($M \in \mathcal{M}^L$) can be answered by only referring to a set of* GMVs $\mathcal{V}$*, if there exists a node v'_t in $\mathcal{G}_\mathcal{V}$ that is L-bisimilar with v_t in $\mathcal{G}_\mathcal{V}$.* □

Example 3: The question in Example 1 can be represented by an inference query $Q(M, v_a, G)$. A set of GMVs $\mathcal{V}$ can be provided, including two virtual GMVs $\mathcal{V}_1$ and $\mathcal{V}_2$, with memory graphs induced from two separately posted sample graphs about Cyclobutadience (C_4H_4) and methylhydroxylamine (CH_5NO), respectively. The extended GMV $\mathcal{V}_1$ sets $\mathcal{T}_{\mathcal{V}_1}$ as two key-value lists that store L-bisimilar node pairs, where keys are nodes in G_1 and values are

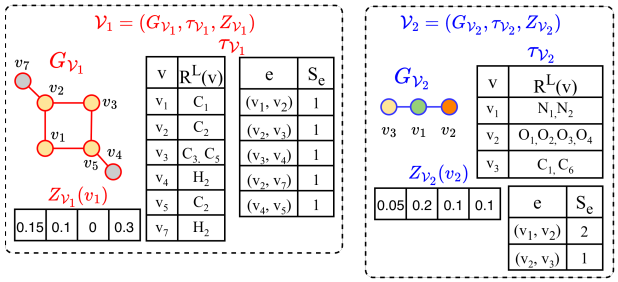


Figure 4: Extended GMVs $\mathcal{V}_1$ and $\mathcal{V}_2$.

Algorithm 2 : GVSel

Input: Graph G, inference query $Q(M, v_t, G)$, a set of views $\mathcal{V}$, a test node v_t, maximum number of views k;
Output: Set of views $\mathcal{V}'$ that best covers $Q(G, M, v)$;

1: set $\mathcal{V}' := \emptyset$, set $U := \emptyset$;
2: $\forall \mathcal{V}_i \in \mathcal{V}$, compute $C[\mathcal{V}_i]$, /*set of nodes in $N^L(v)$ covered*/;
3: **while** $\exists \mathcal{V}_i \in \mathcal{V} \setminus \mathcal{V}' : |S[\mathcal{V}_i] \setminus U| > 0$ or $|\mathcal{V}'| < k$ **do**
4: $\quad \forall \mathcal{V}_i \in V \setminus \mathcal{V}', S[\mathcal{V}_i] := S[\mathcal{V}_i] \setminus U$;
5: $\quad \mathcal{V}^* := \arg\max_{\mathcal{V}_i \in \mathcal{V} \setminus \mathcal{V}'} |S[\mathcal{V}_i]|$ where $S[\mathcal{V}_i] \neq \emptyset$;
6: $\quad \mathcal{V}' := \mathcal{V}' \cup \{\mathcal{V}^*\}$;
7: $\quad U := U \cup S[\mathcal{V}^*]$;
8: **return** $\mathcal{V}'$;

Figure 5: Algorithm GVSel.

nodes in the original G. For example, (v_2, C_2) is a L-bisimilar pair. Z_1 is the precomputed result $M(G_1, v_1)$ that represent inference result of node v_1 over G_1. Similarly for extended GMV $\mathcal{V}_2$. As there exists a 2-bisimilar relation between the union of the two memory graphs ($\mathcal{G}_{\mathcal{V}}$) and $G^2(a_1)$, one can use $\mathcal{V}$ with scaling factors to approximate $Q(M, a_1, G)$ in Example 1. □

Remarks. In practice, a GMV can be constructed to serve the memory graphs induced as subgraphs from published graph data (via procedure L-Bisim) as needed. For example, the memory graphs $G_{\mathcal{V}_1}$ and $G_{\mathcal{V}_2}$ in Fig. 4 are subgraphs of G_1 and G_2. This ensure the inclusion of GMVs having memory graphs with a 2-bisimilar to G if exists, even when their original counterparts may not.

We next investigate three fundamental problems: view selection, view minimization, and a general task of inference using views when the coverage condition may not hold. This gives a comprehensive treatment of our proposed paradigm, providing provable guarantees to justify its feasibility and doability for large graphs.

4 View Selection

For an inference query, one may obtain multiple sample graphs as (virtual) GMVs $\mathcal{V}$ from organizations or data platforms such as Hugging Face [3] and OpenML [36]. To ensure fast inference and reduce *e.g.,* the number of function calls of M inference, we investigate view selection problem.

View selection. Given an inference query $Q(M, v_t, G)$, and a set of GMVs $\mathcal{V}$, we want to find a smallest set $\mathcal{V}' \subseteq \mathcal{V}$, such that $\mathcal{G}'_{\mathcal{V}}$ satisfy the sufficient condition (Lemma 3).

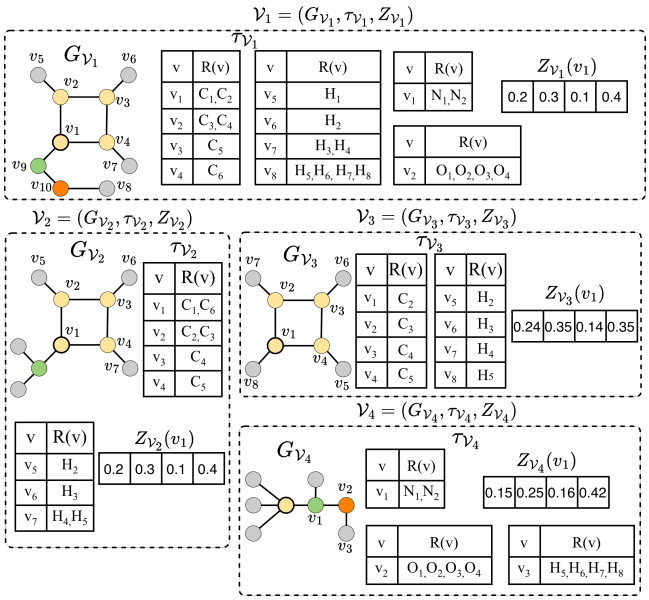


Figure 6: View Selection for Graph Inference. Four views $\mathcal{V}_1$, $\mathcal{V}_2$, $\mathcal{V}_3$, and $\mathcal{V}_4$ are illustrated. $\mathcal{V}_1$ completely covers Q while $\mathcal{V}_2$, $\mathcal{V}_3$, and $\mathcal{V}_4$ covers Q partially.

This problem is not surprisingly NP-hard: a reduction can be constructed from the set cover problem (see Appendix). We next provide an algorithm, denoted as GVSel, for view selection problem.

Selection strategy. Our algorithm restates view selection by testing a "coverage" property of each GMVs. Given the memory graph $G_{\mathcal{V}}$ of a GMV $\mathcal{V}_i \in \mathcal{V}$, it first computes a non-empty l-bisimilar relation R^l between $G^l_{\mathcal{V}}(v_t)$ and $G^l(v_t)$, for $l \in [0, L]$ (by invoking procedure L-Bisim), and with l the largest possible value (up to L). Here a 0-bisimilar relation R^0 exists, if there is a node v'_t in $G_{\mathcal{V}}$ that has the same feature vector as v_t. For each $\mathcal{V}_i$ that contains a node v'_t that is l-bisimilar with v_t, it sets a "cover" set $S[\mathcal{V}_i]$ as the match set of nodes and edges induced by the l-bisimilar relation R^l.

Given an inference query $Q(M, v, G)$ and a set of GMVs $\mathcal{V}$, the algorithm GVSel computes a k-set of GMVs $\mathcal{V}' \subseteq \mathcal{V}$ such that (1) it correctly finds a k-set with a graph $\mathcal{G}_{\mathcal{V}}$ that satisfy the condition in Lemma 3, to allow Q to be exactly answered, or (2) otherwise, it chooses a set of GMVs, such that their memory graphs covers $G^L(v_t)$ via up to maximal l-bisimilar relation, hence with minimum missingness of the nodes and their induced edges that are involved in the computation of $Q(M, v_t, G)$, to approximate the result of Q as close as possible.

Outline. The procedure GVSel first initializes an empty GMV set $\mathcal{V}'$ and an empty coverage set U (line 1). For each candidate view $\mathcal{V}_i \in \mathcal{V}$, GVSel computes $S[\mathcal{V}_i]$ the set of nodes in $N^L(v)$ (line 2). While there are still unselected views that covers uncovered nodes, GVSel conducts the following greedy selection strategy: (1) GVSel updates the uncovered portion of each view by removing nodes already in U (line 4); (2) GVSel selects the view $\mathcal{V}^*$ that covers the largest number of remaining uncovered nodes (line 5); and (3) GVSel adds $\mathcal{V}^*$ to $\mathcal{V}'$ and expand U with the nodes covered by $\mathcal{V}^*$ (line 6-7). This procedure stops when no additional GMVs can be

Algorithm 3 : GVMin

Input: A set of GMVs $\mathcal{V}$, inference query $Q(M, v_t, G)$;
Output: A minimized GMV $\mathcal{V}_m$;

1: set $\mathcal{V}_m := (\mathcal{G}_m, \emptyset, \emptyset)$);
2: $\mathcal{G}_m := \bigcup G_i$ (G_i ranges over memory graphs in $\mathcal{V}$);
3: $\mathcal{P} = \{V_m\}$;
4: Refine $\mathcal{P} = \bigcup_{B \in \mathcal{P}} \text{Refine}(B, \mathcal{P})$ utill convergence;
5: $\mathcal{G}_m := \mathcal{G}_m/\mathcal{P}$, /*collapse each block into equivalence classes/*;
6: set relation $R^I_{\prec} := \emptyset$; compute $R^I_{\prec}$ over $\mathcal{G}_m$;
7: Prune $\mathcal{G}_m$ by removing dominated nodes;
8: Update $\mathcal{V}_m$ with $\mathcal{G}_{\mathcal{V}}$; update $\mathcal{T}_m$ and Z_m accordingly if not $\emptyset$;
9: **return** $\mathcal{V}_m$;

Figure 7: Algorithm GVMin**.**

selected. $\mathcal{V}'$ could achieve a "full coverage" of $G^L(v_t)$ or best effort coverage with some parts that cannot be covered.

Example 4: Suppose M is a 2-layers vanilla GNN, we illustrate GVSel using the views in Fig. 6. For $\mathcal{V}_1$, all nodes in $N^2(v_t)$ have 2-bisimilar matches in G_1 and all edges preserved in $G_{\mathcal{V}_1}$, so $\mathcal{V}_1$ fully covers Q. For $\mathcal{V}_2$, by the definition of 2-bisimulation, one can verify that $\mathcal{V}_2$ covers all the ten nodes on the benzene ring. For $\mathcal{V}_3$, G_3 contains 2-bisimilar carbon nodes in G, so $\mathcal{V}_3$ covers eight nodes. Finally, for $\mathcal{V}_4, G_4$ simulates all the nodes in the $N(OH)_2$ substituent, giving ten matched nodes. Given $\mathcal{V} = \{\mathcal{V}_2, \mathcal{V}_3, \mathcal{V}_4\}$ and $k = 2$, we demonstrate how GVSel selects views. GVSel computes the set of nodes covered by each view in $\mathcal{V}$. In the first iteration of the while loop, GVSel selects one view from the set $\{\mathcal{V}_2, \mathcal{V}_3, \mathcal{V}_4\}$. GVSel selects $\mathcal{V}_2$ since it covers ten nodes more than the other views. Now, set $U := \{S[\mathcal{V}_2]\}$ and $\mathcal{V}' := \{\mathcal{V}_2\}$. In the second iteration, GVSel selects one view from $\{\mathcal{V}_3, \mathcal{V}_4\}$. GVSel selects $\mathcal{V}_4$ since it covers ten uncovered nodes compared to zero uncovered node by $\mathcal{V}_3$. Now, Set $U := \{S[\mathcal{V}_2], S[\mathcal{V}_4]\} := N^L(v)$ and $\mathcal{V}' := \{\mathcal{V}_2, \mathcal{V}_4\}$. Since $|V'| := k$ and $S[\mathcal{V}_3] \setminus U = \emptyset$ ($\mathcal{V}_3$ cannot be selected since it covers zero node), GVSel returns $\mathcal{V}' := \{\mathcal{V}_2, \mathcal{V}_4\}$. Thus, views $\mathcal{V}_2$ and $\mathcal{V}_4$ are selected by GVSel. □

Analysis. GVSel achieves a $(1 - \frac{1}{e})$-approximation guarantee for computing an optimal set of k views with maximum coverage of $G^L(v_t)$. This follows from a reduction from view selection problem to a cardinality constrained maximum weighted set cover problem. GVSel employs a greedy strategy to select the view that covers the largest number of currently uncovered nodes iteratively, until either budget k is reached and no additional views can be selected. The time cost of GVSel is on $O(k|\mathcal{V}|)$ where $|\mathcal{V}|$ is the number of candidate views. Specifically, it takes $O|\mathcal{V}|$ to compute the set of nodes covered by each view initially. Within each iteration, it takes $O(|\mathcal{V}|)$ to update with uncovered nodes for unchoosen views and select the view that covers the largest number of uncovered nodes. Across at most k iterations, this leads to a total runtime of $O(k|\mathcal{V}|)$.

5 View Minimization

While view selection allow us to exploit a small set of reusable GMVs that maximize the coverage of $G^L(v_t)$ with their indistinguishable counterparts, treating inference process as an observer, nodes among the memory graphs themselves may contribute to redundancy and a source of inference and storage overhead, especially for large memory graphs.

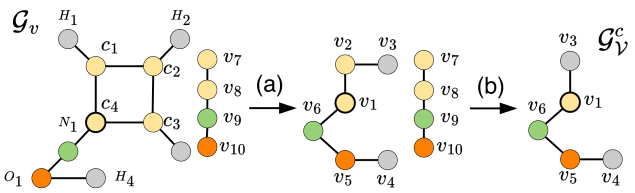


Figure 8: Illustrations of (a) Canonicalization and (b) Pruning.

We next introduce two preprocessing strategies to minimize GMVs. We continue with the coverage property and show that the minimized GMVs has no loss in the nodes in $G^L(v_t)$ they can cover. The first compresses memory graphs to a provable smallest representation (a "canonical form"). The second prunes those that are dominated by other GMVs in terms of coverage ability. We then elaborate with an efficient algorithm to minimize GMVs.

Inference Equivalence. We first show that there is a smallest representation of memory graphs $G_{\mathcal{V}}$, that can be used directly by view-based inference. The idea is to compress $G_{\mathcal{V}}$ with a stronger equivalence condition that preserves the inference indistinguishability in terms of L-bisimilar. To this end, we extend structural equivalence [14] and introduce *inference equivalence relation* $R^I_{\equiv}$.

Given a graph $G_{\mathcal{V}}$, an inference equivalence relation $R^I_{\equiv}$ is a binary relation between the nodes of $G_{\mathcal{V}}$ and nodes in G such that for each node pair $(v, v') \in R^I_{\equiv}$ (1) v and v' has the same input feature vector, and (2) for any neighbor u of v, there exists a neighbor u' of v', such that $(u, u') \in R^I_{\equiv}$.

One can easily verify that $R^I_{\equiv}$ is an equivalence relation. A canonical graph of $G_{\mathcal{V}}$ is the quotient graph $G^c_{\mathcal{V}}$ induced by R_I, which contains a set of nodes, where each node $[v]$ is an equivalent class of a node in $G_{\mathcal{V}}$; and there exists an edge $([v], [v'])$ if (v, v') is an edge in $G_{\mathcal{V}}$. Let $\mathcal{V}_c$ with $G^c_{\mathcal{V}}$ be a *canonicalized* GMV of $\mathcal{V}$, We have the following result.

Lemma 4: *Given an inference query $Q(M, v_t, G)$ with $M \in \mathcal{M}^L$, and a* GMV *$\mathcal{V}$ with memory graph $G_{\mathcal{V}}$, (1) the cover set $S(\mathcal{V}) = S(\mathcal{V}_c)$,* i.e., *the canonical graph $G^c_{\mathcal{V}}$ has no loss in covering $G^L(v_t)$ in terms of L-bisimilarity; and (2) $G^c_{\mathcal{V}}$ is a minimal representation that can preserve the cover set of $\mathcal{V}$.* □

From Lemma 4, the following result follows, justifying that canonical forms of minimized GMVs preserves the inference results.

Corollary 5: *Any inference queries $Q(M, v_t, G)$ with $M \in \mathcal{M}^L$ that can be answered by referring to* GMVs *$\mathcal{V}$ only can also be evaluated with their canonicalized counterpart $\mathcal{V}_c$ alone.* □

These view minimization strategies effectively finds redundancy in view-based graph inference. Such redundancy can be mitigated by a "once-for-all" minimization of $\mathcal{V}$ to a counterpart that covers $G^L(v_t)$, hence in turn reduce inference cost as much as possible.

Inference Dominance. We introduce a second strategy to pre-prune GMVs that are, under no circumstances, cover more than another GMVs for inference query $Q(M, v_t, G)$. To this end, we extend graph simulation [21] for graph inference, denoted as $R^I_{\prec}$.

Given a graph $G_{\mathcal{V}}$ with node set V, $R^I_{\prec} \subseteq V \times V$ is a binary relation such that for any pair $(v, v') \in R^I_{\prec}$, (1) $X^0_v = X^0_{v'}$, *i.e.,* they have the same feature vector, and (2) for any neighbor u of v in $G_{\mathcal{V}}$, there exists a neighbor u' of v' in $G'_{\mathcal{V}}$, such that $(u, u') \in R^I_{\prec}$. In this case, we say v is inference dominated by v', denoted as $v \prec v'$.

One can easily verify that $R^I_{\prec}$ is a preorder (reflexive, transitive), and the maximal relation $R^I_{\prec}$ can be efficiently computed by a slight revision of the algorithms that computes graph simulation [21] in quadratic time. We observe the following.

Lemma 6: *For any inference query $Q(M, v_t, G)$ that can be answered by referring to $\mathcal{V}$ with $\mathcal{G}_{\mathcal{V}}$ only, $Q(M, v_t, G)$ can be answered by $\mathcal{G}_{\mathcal{V}} \setminus \{v\}$, for any node v that is inference dominated by another node v' in $\mathcal{G}_{\mathcal{V}}$ $(v, v') \in R^I_{\prec}$.* □

Algorithm Outline. Given a set of (virtual) GMVs $\mathcal{V}$, the view minimization algorithm, denoted as GVMin, preprocesses $\mathcal{V}$ to a minimal representation $\mathcal{V}_m$ with a compressed memory graph $\mathcal{G}^m_{\mathcal{V}}$ in two phases. (1) The *canonicalization* phase performs a fixpoint, iterative refinement procedure that extends a forward bisimulation [19] with feature equivalence checking. Starting from a coarse partition (line 3), GVMin repeatedly refines blocks by distinguishing nodes with different outgoing-transitions signature until fixpoint stage, *i.e.,*, no further block splits are possible, yielding the coarsest forward bisimulation partition [8, 22] (line 4-6). Each block in the final partition is then collapsed into a node (equivalence class) of $R^I_{\equiv}$, producing the canonicalized graph $\mathcal{G}^m_{\mathcal{V}}$ (line 7). (2) The *pruning* phase next computes the maximum inference dominance relation $R^I_{\prec}$ over $\mathcal{G}^m_{\mathcal{V}}$ to prune the nodes and edges that are dominated by others, further simplify $\mathcal{G}^m_{\mathcal{V}}$ to their minimal representation.

Example 5: We next illustrate the view minimization in Fig. 8. Given $\mathcal{G}_v$ with two disconnected parts: 1) the left contains ten nodes and ten edges; and 2) the right part consists of a chain of four nodes, GVMin initializes partition $\mathcal{P}$ to be the set consisting of only one block V_m where all nodes are grouped to one block. It then iteratively refines and splits the blocks based on the structural equivalence. At the end, $\mathcal{P} = \{(c_1, c_2, c_3), (c_4), (H_1, H_2, H_3), (H_4), (o_1), (N_1), (v_7), (v_8), (v_9), (v_{10})\}$. Hence, the left part is reduced into six supernodes v_1, v_2, v_3, v_4, v_5, and v_6 while the right one is untouched. We then continue with dominance pruning. We observe that node v_7 is dominated by v_2 ($v_7 \prec v_2$) since structural coverage of v_7 can be fully simulated by v_2. One can verify that $v_8 \prec v_1$, $v_9 \prec v_6$, and $v_{10} \prec v_5$. If v_1, v_2, v_5, or v_6 is bisimilar to the test node, v_8, v_7, v_{10}, and v_9 can be pruned since v_1, v_2, v_5, or v_6 can provide an equivalent inference counterpart. Conversely, if v_8, v_7, v_{10}, or v_9 is bisimilar to the test node, inference can rely on v_1, v_2, v_5, or v_6 and its neighbors. This shows that dominance-based pruning removes redundancies while preserving full inference coverage. After the canonicalization and pruning, $\mathcal{G}^c_v$ only consists of five nodes and four edges. □

Analysis. The correctness of GMVInf follows from the correctness of procedures GVMin, GVSel and Lemma 4 and Lemma 6. Let $\mathcal{G}^L(v_t)$ of $\mathcal{V}'_m$ contains $|V_m|$ nodes and $|E_m|$ edges. The construction of $\mathcal{G}_{\mathcal{V}}$ takes $O(k(|V_m| + |E_m|))$ time, and the iterative branching bisimulation partition refinement and inference dominance pruning are in $O(|E_m||V_m|)$. Hence, the total time cost is in $O(k(|V_m| + |E_m|) + |E_m||V_m|)$ per inference query (delay time).

Algorithm 4 : GMVInf (query workload)

Input: Graph G, a set of views $\mathcal{V}$, a set of test nodes V_T, a GNN $M \in \mathbb{M}^L$ with node update function M_v, query workload $W = \{Q_1, Q_2, \ldots, Q_{|V_T|}\}$;
Output: Approximate answer $\mathbf{A}$ to W ;
1: set $\mathbf{A} := \emptyset$;
2: /* preprocessing */
3: $\mathcal{V}_m := \bigcup_{\mathcal{V}_i \in \mathcal{V}} \mathsf{GVMin}(\mathcal{V}_i)$;
4: /* workload processing */
5: **for** $Q_i \in W$ **do**
6: /* select a k-subset $\mathcal{V}'_m$ */
7: $\mathcal{V}'_m := \mathsf{GVSel}(Q_i, \mathcal{V}_m)$;
8: /* view-based inference */
9: **if** $\mathcal{V}'_m \neq \emptyset$ **then**
10: $\mathbf{A} := \mathbf{A} \cup \mathsf{GVInf}(N^L(v_i), \mathcal{V}'_m, M^L)$;
11: **return** $\mathbf{A}$;

Figure 9: Algorithm GMVInf**.**

6 Approximate Inference using Views

We next provide a general algorithm, denoted as GMVInf, to (approximately) evaluate a stream of inference query workload. This extends our discussion to cope with a set of test nodes V_T, and lifts the assumption that the GMVs can be inferred alone.

Algorithm. The algorithm GMVInf processes the workload as a stream of inference queries $Q(M, v_t, G)$, as the test node v_t ranges over a set of test nodes in G.

Preprocessing. GMVInf cold-starts by minimizing a set of GMVs $\mathcal{V}$ with the procedure GVMin to compress each of them into their minimal representation respectively (line 2-3). In this stage, it also invokes the preprocessing component of GVInf to initialize $\mathcal{T}$ and Z, extending any virtual GMVs to extended counterparts, to get it ready to be inferred to directly. This incurs a one-time cost, and the extended GMVs are to be served for the inference workload.

Query Workload Processing. Upon receiving a query workload W where each inference query $Q_i(M, v_i, G)$ with v_i ranges over a stream of test nodes V_T, it invokes the procedure GVSel to select a set of k views $\mathcal{V}'$ that covers as larger part of $G^L(v_i)$ as possible (line 4-6). If the set of selected is not a empty set, it invokes GVInf to compute the inference results for the corresponding test node which approximates the answer of W by rescaling the pre-computed counterparts from $\mathcal{T}$ in the selected minimized GMVs and append the results to A (line 7-9). Finally, it returns the assembled approximate answer A to W (line 10).

To maximize the reusability of GMVs and further reduce the selection cost, algorithm GMVInf dynamically maintains a cache of memory graph following a forgetting principle [35, 39]. The cache periodically swaps out memory graphs that are no longer "fresh" and retain those expected to be needed, determined by GVSel and a

Table 2: Statistics of Datasets (ranked by graph size $|G|$).

Dataset	|V|	|E|	# node types	# attributes
Cora [34]	2,708	5,429	7	1,433
Elliptic++ [12]	5,156	4,784	3	166
ATLAS [4]	59,148	40,823	2	64
Protein [10]	43,471	162,088	2	3
Arxiv [24]	169K	1.2M	40	128
Yelp [51]	717K	7.9M	100	300
Products [24]	2.4M	61.9M	47	100
WS-MAG [23]	0.398B	0.796B	153	768

forgetting curve expressed by an exponential decay model. We provide details and an ablation study of its impact to the performance of view-based inference (see Appendix).

Analysis. GMVInf correctly computes the approximate answer to query load W as guaranteed by (1) the correctness of GVMin to select the set of top k views $\mathcal{V}'$ that best cover each $q \in Q$ with the greedy strategy; (2) the correct computation of the minimized view $\mathcal{V}_m$ process of GVMin to derive a compact single view representation; and (3) the correctness of GVInf to restore the embeddings of original messaging passing through the scaling factors stored in S_e. We next present time cost analysis of GMVInf. For each query in Q, GVSel is in $O(k|\mathcal{V}|)$ and GVMin is in $O(|E^L(v_t)||V^L(v_t)|)$. For each query in Q, the inference with the single minimized view $\mathcal{V}_m$ is in $O(|G_{\mathcal{V}_m}|)$. The memory graph of the minimized view is bounded by $N^L(v)$. The total cost of GMVInf is in $O(|W|(k|\mathcal{V}| + |E^L(v_t)||V^L(v_t)|))$.

7 Experimental Study

Using real-world and benchmark datasets, we evaluate the performance of GMVInf and the impact of its components (view selection GVSel, minimization GVMin, and inference GVInf). Specifically, we evaluate (1) efficiency of our view-based inference methods, in terms of inference cost, and the impact of factors such as query workload size, number of layers, number of GMVs, and graph sizes; (2) accuracy of view-based inference, compared with the original inference with full access to G and M granted, (3) effectiveness of optimization techniques such as forgetting mechanism, and (4) use cases study in real-world graph analytical tasks. Our source code, datasets, and a full version of the paper are made available[2].

7.1 Experimental Settings

Datasets. We employ the following real-world graph benchmark datasets (summarized in Table. 2). (1) **Cora** [34], a citation network where nodes represent documents, and edges are citations. Node features are described by a 0/1-valued word vector indicating the absence/presence of the corresponding word from the dictionary; (2) **Elliptic++** [12] is a Bitcoin transaction dataset, where nodes correspond to wallet addresses (transactions) and temporal edges represent cryptocurrency transfers, with the task of classifying illicit transactions; (3) **ATLAS** [4], is a cybersecurity log dataset used for attack investigation. Each node represents an event (*e.g.*,, system or network activity), and edge denote temporal or causal relationship between events; (4) **Protein** [10], is a molecular graph dataset constructed from 1,113 graphs each representing a protein structure. Nodes correspond to amino acids, and edges indicate spatial proximity or interactions between residues in the 3D conformation. Node features include a 3-dimensional vector encoding properties such as amino acid type, degree of hydrophobicity, and secondary structure role; (5) **Arxiv** [24], a citation network with nodes representing arXiv papers and edges denoting one paper cites another one. The features of each node includes a 128-dimensional feature vector obtained by averaging the embeddings of words in its title and abstract; (6) **Yelp** [51] is prepared from the raw json data of businesses, comprising a dense network of user-business interactions. Nodes represent users and edges are generated if two users are friends. Node features are 300-dimensional the word embeddings derived by reviews of products; (7) **Products** [24], a large-scale product co-purchase network, where nodes represent products sold in Amazon and edges denote products purchased together.

Besides real-world datasets, we also generated a large synthetic dataset **WS-MAG**, extending the core of the real-world MAG240M citation network [23] using a small-world generator [48]. **WS-MAG** enables controlled studies of the impact of graph size $|G|$ for graphs upto billion-scale. Specifically, we vary $|G|$ from 6 million to 1.194 billion while fixing the average node degree to be 2.

Graph models. We have pre-trained three classes of representative GNNs: GCNs[3] [27], GATs[3] [44], and GraphSAGE[3] [20], for each dataset. For a fair comparison, (1) for all the datasets, we consider the task of node classification, and (2) all methods (when compared against each other) are applied for the same set of GNNs.

Baselines. We compare our algorithm GVInf against the following baselines: (1) GInf, direct inference using original graph G without views; (2) GVInf-all, a variant of GVInf where GVSel refers to all views without forgetting mechanism; (3) GVInf-ran, a variant of GVInf that replaces GVSel with a random selection of k views from $\mathcal{V}$; (4) GVInf-nomin, a variant of GVInf without view minimization. (5) DSpar [32], state-of-the-art graph compression method, performing edge down-sampling to preserve graph spectral information to accelerate GNN inference.

Evaluation Metrics. We evaluate GVInf, its variants, and baselines with the following measures. (1) Efficiency. (a) Inference Speed-up. We define the speed-up of inference as $\frac{T_{inf}}{T_{oinf}}$, where T_{inf} is the inference time cost of GInf over G, and T_{oinf} represents the inference time cost of view-based and baseline counterparts. (b) Normalized Compression Ratio (ncr). This measures how well (minimized) views reduce the amount of data to be accessed, compared to the size of L-hops subgraph of the test node. It is defined as $\text{ncr} = 1 - \frac{|\mathcal{V}_m|}{|G^L|}$; the closer to 1, the better. (2) Effectiveness. We report the accuracy of GInf and GVInf variants. For Yelp where the benchmark task is a multi-class node classification, we report micro $F1$-score. To ensure a fair evaluation, we report the average performance of 50 different batches of query workload having the same size.

7.2 Experimental Results

Exp-1: Efficiency of View-based Inference. We present the first set of experiments evaluating inference efficiency measured by inference speed-up for GVInf, its variants, and DSpar relative to GInf. In particular, we examine how the efficiency of GVInf, its variants, GInf, and DSpar varies with key impact factors, including

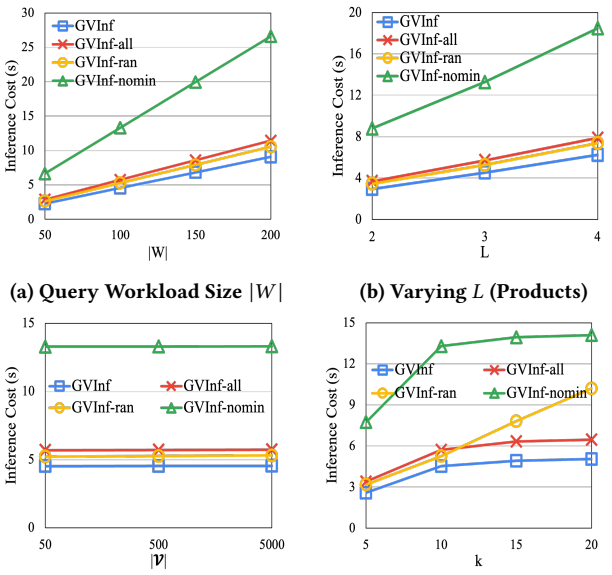
[2]https://anonymous.4open.science/r/GMV-E6A6

Table 3: Inference Speed-up and Normalized Compression Ratio ncr. ncr **are marked in parentheses. Best performance is highlighted in bold. Inference speed-up is computed relative to** GInf. ncr **is computed relative to $G^L(v_t)$ of test node v_t.**

	Cora	Proteins	Arxiv	Yelp	Products
GVInf	**5.35 (0.78)**	**41.85 (0.76)**	**80.19 (0.81)**	**138.82 (0.82)**	**849.98 (0.89)**
GVInf-all	4.57 (0.74)	37.48 (0.73)	68.65 (0.78)	114.13 (0.78)	694.81 (0.87)
GVInf-ran	4.66 (0.74)	38.71 (0.74)	64.68 (0.77)	125.41 (0.80)	739.29 (0.88)
GVInf-nomin	2.62 (0.54)	21.53 (0.54)	36.39 (0.59)	61.62 (0.59)	291.01 (0.69)
Dspar	1.57 (0.24)	15.9 (0.37)	23.3 (0.36)	46.5 (0.46)	124.18 (0.28)

Table 4: Inference Accuracy: Overall Comparison. Best performance is highlighted in bold.

	Cora	Protein	Arxiv	Yelp	Products
GVInf	0.7966 ± 0.0767	0.7356 ± 0.0741	0.7148 ± 0.0653	0.6514 ± 0.0712	**0.8031 ± 0.0693**
GVInf-all	**0.8055 ± 0.0604**	0.7323 ± 0.0684	0.7092 ± 0.0614	0.6581 ± 0.0679	0.8017 ± 0.0637
GVInf-ran	0.6722 ± 0.0831	0.6256 ± 0.0815	0.6448 ± 0.0741	0.5531 ± 0.0786	0.6879 ± 0.0752
GVInf-nomin	0.7964 ± 0.0735	0.7302 ± 0.0726	0.7153 ± 0.0656	0.6523 ± 0.0708	0.7985 ± 0.0611
DSpar	0.7542 ± 0.0794	0.7165 ± 0.0783	0.6875 ± 0.0679	0.6124 ± 0.0773	0.7567 ± 0.0786
GInf	0.7971 ± 0.0121	**0.7387 ± 0.0147**	**0.7175 ± 0.0103**	**0.6592 ± 0.0194**	0.8017 ± 0.0258



(a) Query Workload Size $|W|$ **(b) Varying L (Products)**

(c) Varying $|\mathcal{V}|$ (Products) **(d) Varying k (Products)**

Figure 10: Inference Costs of View-based Inference.

query workload size $|W|$, number of layers L, number of views $|\mathcal{V}|$, and maximum number of selected views k.

Inference Speed-up: Overall Performance. Fixing the number of views $|\mathcal{V}|$ being 500, query loads size $|W| = 100$, maximum number of views being 10, GNNs class to be a 3-layers GCN, we compare the inference speed-up of GVInf, the variants of GVInf, and DSpar over GInf across **Cora**, **Protein**, **Arxiv**, **Yelp**, and **Products** datasets. We report the average inference speed-up and normalized compression ratio (ncr) over all test nodes under a query Q in Table. 3.

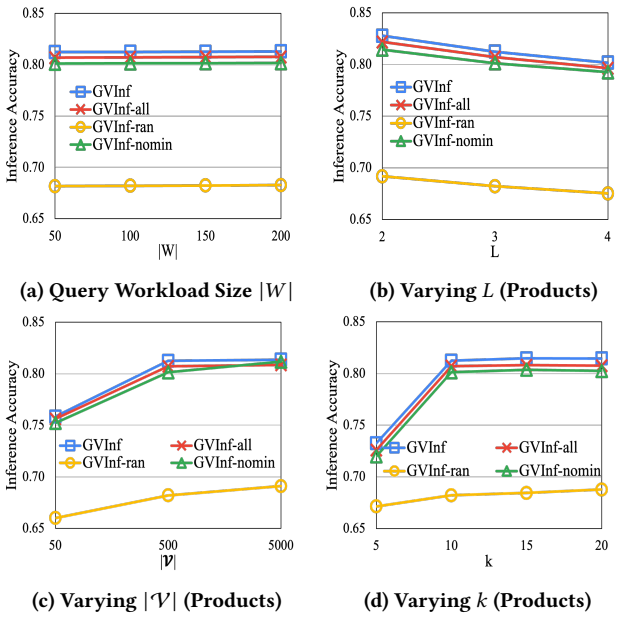
We observe the following. (1) GVInf significantly improves graph inference with a speed-up from at least ×**5** to ×**850**. Indeed, the larger the graph is, the better GMVs-based methods improve GInf. (2) GVInf achieves a greater speed-up in all cases. GVInf-all and GVInf-ran performs better than GVInf-nomin, due to minimization and selection strategies. (3) GVInf and the variants outperform DSpar, due to their learning-free and model agnostic process.

Varying Query Workloads Size $|W|$. Fixing $|\mathcal{V}|$ being 500, k being 10, GNNs class to be a 3-layers GCN, we vary $|W|$ from 50 to 200. As illustrated in Fig. 10(a), we find that (1) the inference costs of GVInf and its variants increase linearly as $|W|$ increases; and (2) GVInf achieves the lowest inference costs while GVInf-nomin incurs the highest which is consistent to the observation of the low inference speed up achieved by GVInf-nomin in Table. 3.

Varying Number of Layers L. Fixing $|\mathcal{V}|$= 500, $|W|$= 100, k= 10, we vary the L of GCN from 2 to 4. As shown in Fig. 10(b), the inference cost of GVInf and its variants increase linearly as the number of layers increase from 2 to 4. We remark that this result include once-for-all overhead on extending virtual GMVs, for which we have consistent observation with GNN inference cost analysis in [47].

Varying Number of Views $|\mathcal{V}|$. Fixing $|W|$ being 100, GNN class being a 3-layers GCN, maximum number of views k being 10, we vary the number of views $|\mathcal{V}|$ from 50 to 5,000. Based on Fig. 10(c), we find that inference cost of GVInf and its variants remain insensitive as $|\mathcal{V}|$ increases. This is because GVInf and variants incur inference cost that is determined by the size of selected GMVs (which is always 10 regardless of how large $|\mathcal{V}|$ is) and GNN classes only, and benefit from fast view selection.

Varying k (# of selected GMVs*).* Fixing $|Q|$ being 100, the GNN class being a 3-layers GCN, $|\mathcal{V}|$ being 500, we vary the maximum number of views k from 5 to 20 to examine how k affects the inference time



(a) Query Workload Size $|W|$ (b) Varying L (Products)

(c) Varying $|\mathcal{V}|$ (Products) (d) Varying k (Products)

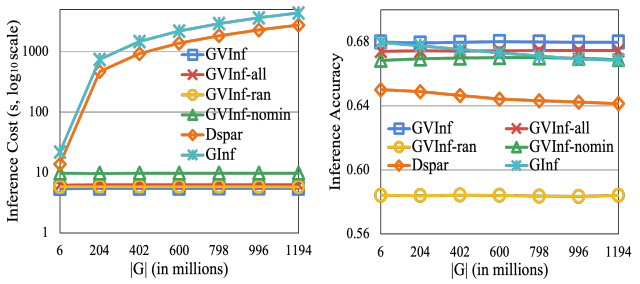
Figure 11: Inference Accuracy of View-based Inference.

cost of GVInf and the variants of GVInf on ***Products***. As shown in Fig. 10(d), when k increases from 5 to 20, the inference costs of GVInf, GVInf-all, and GVInf-nomin all increase first and then gradually level off. This trend occurs because the actual number of selected views actually saturated after k exceeds 10. No more views are selected as k keeps increasing after a certain point. By contrast, inference cost of GVInf-ran increases linearly, as it selects exactly k views, leading to a linear increase in cost with respect to k.

Exp-2: Accuracy of View-based Inference. We next evaluate inference accuracy measured by F1-score (resp. micro F1-score) of single node classification (resp. multi-class node) classification. In particular, we examine how the inference accuracy of GVInf, the variants of GVInf, DSpar, and GInf change with varying key impact factors: query workload size $|W|$, number of layers L, number of views $|\mathcal{V}|$, and maximum number of selected views k.

Inference Accuracy. Fixing $|\mathcal{V}|$ being 500, $|Q| = 100$, k being 10, GNNs class to be a 3-layers GAT, we compare the inference accuracy of GVInf and the variants of GVInf across **Cora**, **Protein**, **Arxiv**, **Yelp**, and ***Products*** in Table. 4. We observe the followings: (1) Inference accuracy achieved by GVInf, GVInf-all, GVInf-nomin is generally comparable to each other across all datasets. GVInf, GVInf-all, and GVInf-nomin incurs negligible or no loss relative to GInf across all datasets; and (2) GVInf and its variants outperform DSpar, except for GVInf-ran. This result is expected since randomly selected views by GVInf-ran tend to provide poor coverage of Q, leading to reduced inference accuracy compared to GVInf.

Varying Query Workloads Size $|W|$. Fixing $|\mathcal{V}|$ being 500, k being 10, GNNs class to be a 3-layers GCN, we vary $|W|$ from 50 to 200 to compare inference accuracy of GVInf and the variants of GVInf over ***Products***. As shown in Fig. 11(a), we observe the followings. First, the inference accuracy of GVInf, GVInf-nomin, and GVInf-all remain stable and comparable to each other as $|W|$ increases.



(a) Inference Cost (b) Inference Accuracy

Figure 12: The Impact of $|G|$ on View based graph inference.

Inference accuracy is insensitive to the number of test nodes since the test nodes are randomly selected. Second, GVInf-ran incurs the lowest inference accuracy which is consistent to the observation of the low inference accuracy achieved by GVInf-ran in Table. 4 caused by the poor coverage by randomly selected views.

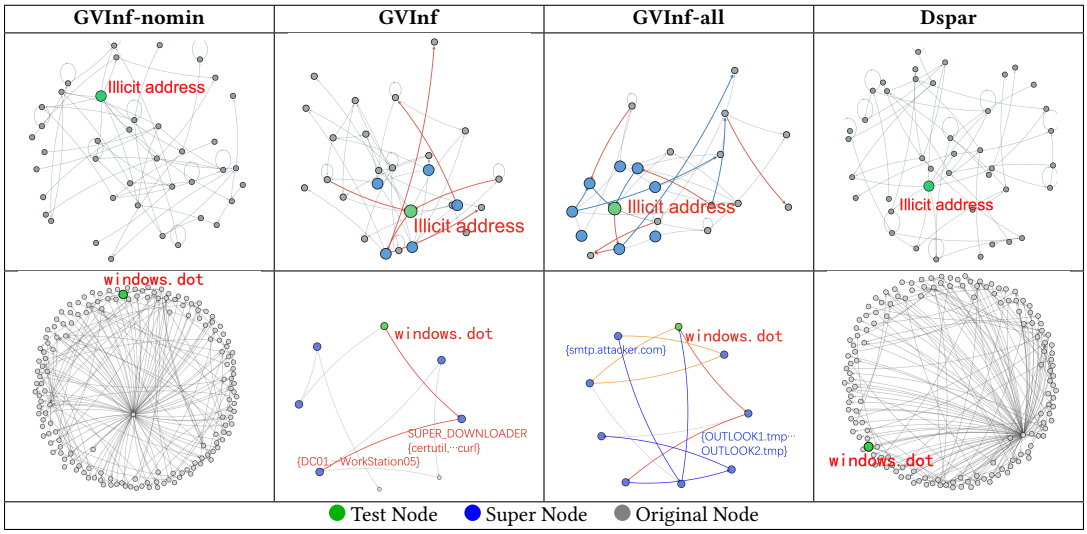
Varying Number of Layers L. Fixing $|\mathcal{V}|$ being 500, k being 10, $|W|$ being 100, we vary L of GCN from 2 to 4 layers to evaluate how the number of layers affect the performance of GVInf and the variants of GVInf over ***Products***. As shown in Fig. 11(b), the inference accuracy of GVInf and its variants drops as the L increases from 2 to 4. This observation is consistent to the oversmoothing issue encountered by GCN where nodes representation starts to look similar and becomes indistinguishable after multiple layers [38].

Varying the Number of Views $|\mathcal{V}|$. Fixing $|W|$ being 100, k being 10, GNN class being a 3-layers GCN, we vary $|\mathcal{V}|$ from 50 to 5,000. As shown in Figure. 11(c), the inference accuracy of GVInf and its variants improves consistently as $|\mathcal{V}|$ increases, but gradually saturates once $|\mathcal{V}|$ exceeds 500. This saturation occurs because, beyond a certain threshold, the top-k views are sufficient to cover most of the inference query Q. Notably, GVInf-ran has much lower inference accuracy compared to GVInf, GVInf-all, and GVInf-nomin.

Varying the Maximum Number of Selected Views k. Fixing the query loads $|W|$ being 100, GNN class being a 3-layers GCN, the total number of views $|\mathcal{V}|$ being 500, we vary maximum number of views k from 5 to 20. We observe the followings from Fig. 11(d). (1) As k increases from 5 to 20, the inference accuracy of GVInf, GVInf-all, and GVInf-nomin all increase first and then gradually level off. This trend occurs because the coverage of inference query is saturated after k exceeds 10. No more views are selected as k keeps increasing, leading to a steady performance after k reaches a certain threshold; and (2) GVInf-ran exhibits minor accuracy improvements as k increases, because it strictly selects k views, which slightly enhances but not substantially expands the coverage of inference query Q.

Exp-3: Impact of $|G|$. Fixing $|Q|$ being 100, $|\mathcal{V}|$ being 500, k being 10, GNNs class being a 3-layers GAT, utilizing the simulated graph dataset **WS-MAG**, we vary the size of graph $|G|$ from 6 million to 1.194 billion to study how graph size affects inference time cost and inference accuracy of GVInf, the variants of GVInf compared to GInf and DSpar. (1) As $|G|$ increases from 6 million to 1.194 billion, the inference costs of GVInf, the variants of GVInf, DSpar, and GInf follow the patterns from Fig. 12(a). The inference cost of DSpar and GInf increases as $|G|$ increases. The inference cost of

Table 5: Case Study: Bitcoin Laundering and Document Exploit.



GVInf and its variants remain stable regardless of the graph size, as their inference cost are determined by the size of GMVs $|\mathcal{V}|$. (2) The inference accuracy of GInf and its variants remain stable (Fig. 12(b)). Interestingly, the accuracy of DSpar and GInf even drops slightly. We found that this is due to the output quality of view-based inference also only depends on memory graphs, while DSpar and GInf may suffer accuracy loss due to noisy messages and over-smoothing in large graphs [32, 38].

Exp-4: Ablation Analysis. We also conducted ablation analysis to evaluate the effectiveness of selection, minimization and forgetting mechanism to the accuracy, inference cost, and memory cost, respectively. While each contributed positively to all the measures, GVSel has most significant contribution to accuracy (14.17% loss if removed); and GVMin has contributed most on reducing memory cost and inference cost. We report more details in the appendix.

Summary. In general, we found that view-based graph inference improve direct inference over large G (at million-billon scale; including **Arxiv**, **yelp**, **Products** and **WS-MAG**) by 488 times on average, accessing only 15.7% of original data, with on average only 0.46% loss of accuracy. These suggest promising application of GMV-based approaches for large-scale graph analysis.

Exp-5: Case Studies. We next showcase use cases of GVInf, in cybersecurity [4] and Bitcoin network analysis [12].

Case Study I: Bitcoin Money Laundering Detection. Anomaly detection in large Bitcoin transaction networks trains GNN-based classifiers to detect whether an account (IP address) is "illicit" or not. We identify two published, small fraction of networks that justify known money laundry patterns such as *Peel Chain* (self-loops) and *Spindle* (disperse–converge) [26], as GMVs (Table 5). (1) All of our GMV-based methods (GVInf-all,GVInf-nomin,GVInf) correctly detected illcit accounts, consistent with the results reported by GInf. (2) With minimization strategies, GVInf is able to achieve smallest cost, involving at most 5 "super nodes" (equivalent classes) of relaying IP accounts that play the same role, with inference accuracy 0.78, and a ×**7** times improvement of GInf (inference with full access of entire networks). GVInf-all explores larger GMVs with more nodes (ncr **0.78**, inference accuracy **0.81**); and GVInf-nomin involves most nodes in GMVs. DSpar requires a full access of entire network to learn a sparse structure (only a fraction is shown).

Case Study II: Attack pattern detection. Cybersecurity analysts may train GNNs to predict links in system log graphs to retrieve attack patterns [4] with malicious access to files (*e.g.,* "windows.dot", Table 5 (Row 2)). Three attack patterns that takes confirmed vulnerabilities are posted and wrapped as GMVs: $\mathcal{V}_1$ (*"Email Delivery":*) (smtp.attacker.com → outlook.exe → ··· → windows.dot), $\mathcal{V}_2$ (*"Exploitation":*) (windows.dot → dot.exe → cmd.exe → certutil → ··· → beacon.dll), and $\mathcal{V}_3$ (*"Propagation":*) (windows.dot → psexec → net → DC01 → ··· → WorkStation05). While GVInf-nomin explores all three GMVs with an inference over entire 3-hop subnetwork, GVInf can automatically merge equivalent handlers (files):{certutil,wget,curl}:SUPER_DOWNLOADER, yielding **5** supernodes (ncr **0.93**, inference accuracy **0.72**) and incurs smallest inference cost, improving GInf by × **140** times.

8 Conclusion

We have proposed graph inference using views, a novel paradigm that scales graph inference by serving and referring to graph model views (GMVs) to approximate the inference outputs, with reduced time and storage costs. GMVs package graph models, test data and results as queryable structures. We formally investigated three core problems, view selection, view minimization, and inference querying using views, establishing their theoretical properties and introducing efficient algorithms for each. Our experiments on benchmark datasets demonstrated that GMVs achieves ×5 - ×850 inference speed-ups, accessing on average 15.7% of the original data for large graphs, and with little loss of the inference accuracy. The study of view-based graph inference is a first step. A future topic is to extend GMVs to support inference analysis for dynamic graphs.

References

[1] Sergio Abriola, Pablo Barceló, Diego Figueira, and Santiago Figueira. 2018. Bisimulations on data graphs. *Journal of Artificial Intelligence Research* 61 (2018), 171–213.

[2] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *VLDB 2000, Proceedings of the 26th International Conference on Very Large Data Bases*. Morgan Kaufmann, 496–505.

[3] Hugging Face AI. 2025. Hugging Face – The AI Community Building the Future. Retrieved Apr 13, 2025 from https://huggingface.co/

[4] Abdulellah Alsaheel, Yuhong Nan, Shiqing Ma, Le Yu, Gregory Walkup, Z Berkay Celik, Xiangyu Zhang, and Dongyan Xu. 2021. {ATLAS}: A sequence-based learning approach for attack investigation. In *30th USENIX Security Symposium (USENIX Security 21)*. 3005–3022.

[5] Youhui Bai, Cheng Li, Zhiqi Lin, Yufei Wu, Youshan Miao, Yunxin Liu, and Yinlong Xu. 2021. Efficient data loader for fast sampling-based GNN training on large graphs. *IEEE Transactions on Parallel and Distributed Systems* 32, 10 (2021), 2541–2556.

[6] Pablo Barceló, Egor V Kostylev, Mikaël Monet, Jorge Pérez, Juan L Reutter, and Juan-Pablo Silva. 2020. The expressive power of graph neural networks as a query language. *SIGMOD Record* (2020).

[7] Maciej Besta and Torsten Hoefler. 2024. Parallel and distributed graph neural networks: An in-depth concurrency analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 5 (2024), 2584–2606.

[8] Amar Bouali and Robert De Simone. 1992. Symbolic bisimulation minimisation. In *International Conference on Computer Aided Verification*. 96–108.

[9] Zhenkun Cai, Xiao Yan, Yidi Wu, Kaihao Ma, James Cheng, and Fan Yu. 2021. DGCL: An efficient communication library for distributed GNN training. In *Proceedings of the Sixteenth European Conference on Computer Systems*. 130–144.

[10] Gaoxiang Chen, Liya Hou, Zhanwei Li, Bin Xie, and Yongqiang Liu. 2025. A new strategy for Cas protein recognition based on graph neural networks and SMILES encoding. *Scientific Reports* 15, 1 (2025), 15236.

[11] Ming Chen, Zhewei Wei, Bolin Ding, Yaliang Li, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. 2020. Scalable graph neural networks via bidirectional propagation. *Advances in neural information processing systems* 33 (2020), 14556–14566.

[12] Youssef Elmougy and Ling Liu. 2023. Demystifying fraudulent transactions and illicit nodes in the bitcoin network for financial forensics. In *SIGKDD*. 3979–3990.

[13] Wenfei Fan, Xin Wang, and Yinghui Wu. 2014. Answering graph pattern queries using views. In *2014 IEEE 30th International Conference on Data Engineering*. IEEE, 184–195.

[14] Yangxin Fan, Haolai Che, and Yinghui Wu. 2025. Inference-friendly Graph Compression for Graph Neural Networks. *arXiv preprint arXiv:2504.13034* (2025).

[15] Matthias Fey and Jan Eric Lenssen. 2019. Fast graph representation learning with PyTorch Geometric. *arXiv preprint arXiv:1903.02428* (2019).

[16] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[17] Matt Gardner, Joel Grus, Mark Neumann, Oyvind Tafjord, Pradeep Dasigi, Nelson Liu, Matthew Peters, Michael Schmitz, and Luke Zettlemoyer. 2018. Allennlp: A deep semantic natural language processing platform. *arXiv preprint arXiv:1803.07640* (2018).

[18] Floris Geerts. 2023. A Query Language Perspective on Graph Learning. In *PODS*.

[19] Jan Friso Groote, David N. Jansen, Jeroen J. A. Keiren, and Anton J. Wijs. 2017. An O(mlogn) Algorithm for Computing Stuttering Equivalence and Branching Bisimulation. *ACM Trans. Comput. Logic* 18, 2 (2017).

[20] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *NeurIPS* (2017).

[21] Monika Rauch Henzinger, Thomas A Henzinger, and Peter W Kopke. 1995. Computing simulations on finite and infinite graphs. In *Proceedings of IEEE 36th Annual Foundations of Computer Science*. 453–462.

[22] Johanna Högberg, Andreas Maletti, and Jonathan May. 2007. Bisimulation minimisation for weighted tree automata. In *International Conference on Developments in Language Theory*. 229–241.

[23] Weihua Hu, Matthias Fey, Hongyu Ren, Maho Nakata, Yuxiao Dong, and Jure Leskovec. 2021. OGB-LSC: A Large-Scale Challenge for Machine Learning on Graphs. *NeurIPS* (2021).

[24] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. *NeurIPS* (2020).

[25] Wenke Huang, Guancheng Wan, Mang Ye, and Bo Du. 2024. Federated graph semantic and structural learning. *arXiv preprint arXiv:2406.18937* (2024).

[26] George Kappos, Haaroon Yousaf, Rainer Stütz, Sofia Rollet, Bernhard Haslhofer, and Sarah Meiklejohn. 2022. How to peel a million: Validating and expanding bitcoin clusters. In *31st usenix security symposium (usenix security 22)*. 2207–2223.

[27] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[28] Dongyue Li, Tao Yang, Lun Du, Zhezhi He, and Li Jiang. 2021. Adaptivegcn: Efficient gcn through adaptively sparsifying graphs. In *CIKM*. 3206–3210.

[29] Jia Li, Yang Cao, and Xudong Liu. 2016. Approximating graph pattern queries using views. In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*. 449–458.

[30] Jinfei Liu, Jian Lou, Junxu Liu, Li Xiong, Jian Pei, and Jimeng Sun. 2021. Dealer: An end-to-end model marketplace with differential privacy. *Proceedings of the VLDB Endowment* 14, 6 (2021).

[31] Tianfeng Liu, Yangrui Chen, Dan Li, Chuan Wu, Yibo Zhu, Jun He, Yanghua Peng, Hongzheng Chen, Hongzhi Chen, and Chuanxiong Guo. 2023. {BGL}:{GPU-Efficient}{GNN} training by optimizing graph data {I/O} and preprocessing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 103–118.

[32] Zirui Liu, Kaixiong Zhou, Zhimeng Jiang, Li Li, Rui Chen, Soo-Hyun Choi, and Xia Hu. 2023. Dspar: An embarrassingly simple strategy for efficient gnn training and inference via degree-based sparsification. *Transactions on Machine Learning Research* (2023).

[33] Jan Martens, Jan Friso Groote, Lars van den Haak, Pieter Hijma, and Anton Wijs. 2021. A linear parallel algorithm to compute bisimulation and relational coarsest partitions. In *International Conference on Formal Aspects of Component Software*. 115–133.

[34] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. 2000. Automating the construction of internet portals with machine learning. *Information Retrieval* (2000).

[35] Jaap MJ Murre and Joeri Dros. 2015. Replication and analysis of Ebbinghaus' forgetting curve. *PloS one* 10, 7 (2015), e0120644.

[36] OpenML. 2025. OpenML: A worldwide machine learning lab. https://openml.org/

[37] Jian Pei, Raul Castro Fernandez, and Xiaohui Yu. 2023. Data and AI model markets: Opportunities for data and model sharing, discovery, and integration. *Proceedings of the VLDB Endowment* 16, 12 (2023), 3872–3873.

[38] T Konstantin Rusch, Michael M Bronstein, and Siddhartha Mishra. 2023. A survey on oversmoothing in graph neural networks. *arXiv preprint arXiv:2303.10993* (2023).

[39] Sourav S Bhowmick, SH Annabel Chen, and Divesh Srivastava. 2024. Cognitive Psychology Meets Data Management: State of the Art and Future Directions. In *SIGMOD*.

[40] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE transactions on neural networks* 20, 1 (2008), 61–80.

[41] Yingxia Shao, Hongzheng Li, Xizhi Gu, Hongbo Yin, Yawen Li, Xupeng Miao, Wentao Zhang, Bin Cui, and Lei Chen. 2024. Distributed graph neural network training: A survey. *Comput. Surveys* 56, 8 (2024), 1–39.

[42] Qi Song, Yinghui Wu, Peng Lin, Luna Xin Dong, and Hui Sun. 2018. Mining summaries for knowledge graph search. *IEEE Transactions on Knowledge and Data Engineering* 30, 10 (2018), 1887–1900.

[43] Petar Veličković. 2023. Everything is connected: Graph neural networks. *Current Opinion in Structural Biology* 79 (2023), 102538.

[44] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat* (2017).

[45] Cheng Wan, Youjie Li, Cameron R Wolfe, Anastasios Kyrillidis, Nam Sung Kim, and Yingyan Lin. 2022. Pipegcn: Efficient full-graph training of graph convolutional networks with pipelined feature communication. *arXiv preprint arXiv:2203.10428* (2022).

[46] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. 2019. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315* (2019).

[47] Zhaokang Wang, Yunpan Wang, Chunfeng Yuan, Rong Gu, and Yihua Huang. 2021. Empirical analysis of performance bottlenecks in graph neural network training and inference with GPUs. *Neurocomputing* 446 (2021), 165–191.

[48] Duncan J Watts and Steven H Strogatz. 1998. Collective dynamics of 'small-world'networks. *nature* (1998).

[49] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).

[50] Jian Yang, Kamalakar Karlapalem, and Qing Li. 1997. Algorithms for Materialized View Design in Data Warehousing Environment. In *VLDB'97, Proceedings of the 23rd International Conference on Very Large Data Bases*. Morgan Kaufmann, 136–145.

[51] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2019. Graphsaint: Graph sampling based inductive learning method. *arXiv preprint arXiv:1907.04931* (2019).

[52] Chao Zhang, Jiaheng Lu, Qingsong Guo, Xinyong Zhang, Xiaochun Han, and Minqi Zhou. 2021. Automatic view selection in graph databases. In *Proceedings of the 33rd International Conference on Scientific and Statistical Database Management*. 197–202.

[53] Cheng Zheng, Bo Zong, Wei Cheng, Dongjin Song, Jingchao Ni, Wenchao Yu, Haifeng Chen, and Wei Wang. 2020. Robust graph representation learning via neural sparsification. In *International conference on machine learning*. PMLR,

11458–11468.
[54] Da Zheng, Chao Ma, Minjie Wang, Jinjing Zhou, Qidong Su, Xiang Song, Quan Gan, Zheng Zhang, and George Karypis. 2020. DistDGL: Distributed graph neural network training for billion-scale graphs. In *2020 IEEE/ACM 10th Workshop on Irregular Applications: Architectures and Algorithms (IA3)*. IEEE, 36–44.
[55] Da Zheng, Xiang Song, Chengru Yang, Dominique LaSalle, and George Karypis. 2022. Distributed hybrid cpu and gpu training for graph neural networks on billion-scale heterogeneous graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4582–4591.
[56] Hongkuan Zhou, Ajitesh Srivastava, Hanqing Zeng, Rajgopal Kannan, and Viktor Prasanna. 2021. Accelerating large scale real-time GNN inference using channel pruning. *arXiv preprint arXiv:2105.04528* (2021).

9 Appendix

9.1 Appendix A: Notations and Proofs

Proof of Corollary 3. Given a set of GMVs $\mathcal{V} = \{\mathcal{V}_1, \ldots, \mathcal{V}_n\}$, we show that GVInf can be readily applied with a preprocessing. It first extends all the virtual views in $\mathcal{V}$ if any. It then creates a single GMV $\mathcal{V}'$ with a single memory graph $\mathcal{G}_{\mathcal{V}}$ as the union of the memory graph $G_{\mathcal{V}_i}$ in each GMV $\mathcal{V}_i$. The processing of Q follows its counterpart GVInf. The only difference is that when updating the inference results, it traces back to the individual $\mathcal{T}_{\mathcal{V}_i}$ and Z_v of the node v in $G_{\mathcal{V}_i}$ to derive the scalars from v's local neighborhood. The correctness follows from the analysis of GVInf.

Hardness of view selection problem. We verify the NP-hardness of view selection problem with a reduction from the set cover problem. An instance of set cover problem (a known NP-complete problem) has a set of elements S, and a set $\mathcal{S}$ of subsets of S. The goal is to find a subset $\mathcal{S}' \subseteq \mathcal{S}$ with size at most k, such that $S = \bigcup_{S_i \in \mathcal{S}'} S_i$. Our reduction is constructed as follows. (1) For each set $S_i \in \mathcal{S}$, We construct a memory graph $G_{\mathcal{V}}$ as a two-level tree with a root v_t and an initial feature vector X_{v_t}. For each element $s_i \in S$, an edge $e_i = (v_t, v_i)$, where v_i is a new node with a unique feature vector assigned to s_i. Similarly, for each subset $S_j \in \mathcal{S}$, a single root tree T_j is constructed, with (a) a root node v_{t_j} with the same feature vector $X_{v_{t_j}} = X_{v_t}$, and (b) an edge $e_{jk} = (v_{t_j}, v_{jk})$. We initialize a Vanilla GNN with 2 layers, which simply assigns all the nodes v_{t_j} and the node v_t a label l. Given a cardinality constraint b for view selection problem, and set $b = k$. This construction is in polynomial time. This generates a set of $|\mathcal{S}|$ GMVs $\mathcal{V}$, where each GMV $\mathcal{V} = (T_j, \emptyset, \emptyset)$. We can verify that there exists a set cover of the set S that contains k subsets from $\mathcal{S}$, if and only if a size-k set $\mathcal{V}'$ with a forest $\mathcal{G}_{\mathcal{V}}$ as the union of trees T_j from each GMV $\mathcal{V}_i \in \mathcal{V}'$, that ensures to have at least v_i that is bisimilar to v_t.

Proof of Corollary 5. It suffices to show the following two statements. (1) For any set of GMVs $\mathcal{V}$ with $\mathcal{G}_{\mathcal{V}}$ having a node v'_t that is L-bisimilar to v_t, the canonicalized GMVs $\mathcal{V}_c$ also has a counterpart $[v'_t]$ in $\mathcal{G}_{[}\mathcal{V}]$. This can be shown by observing that for any node pair $(v, v') \in R^L$ and any node pair $(v', v'') \in R_I$ (with v' and v'' in $G_{\mathcal{V}}$), $(v, v'') \in R^L$. (2) To reconstruct $Q(M, v_t, G)$ from the cononicalized GMVs, we only need to update the scalors by a proper calibaration in terms of the neighborhood information over the nodes $[v']$ in $G^c_{\mathcal{V}}$, and the rest processing remain intact.

Proof of Lemma 6. For any nodes and edges participating in the inference computation of v, there exists at least a counterpart in the neighbors of v' that contribute to the computation of v'. If v' is L-bismilar to a test node v_t in an inference query, v contributes no more than v_t in terms of message propagation, hence can be pruned. While if v is L-similar to v_t, then v' always have a counterpart that is L-similar to v_t, induced by $R^I_{\prec}$. Hence, given a set of canonicalized GMVs $\mathcal{V}^c$, we can safely prune all the nodes and their edges that are inference dominated by others via $R^I_{\prec}$.

9.2 Appendix B: Algorithms and Analysis

A Parallel Implementation of GVMin. Given sufficient computational resources (*i.e.,* multi-cores CPUs or GPUs), GVMin can be accelerated by adopting the Par-BCRP Algorithm [33], the first known linear-time method for computing strong bisimulation via relational coarsest partitioning. By distributing up to $n = max\{|V^L|, |E^L|\}$ parallel processors, GVMin can parallelize the workloads of iterative branching bisimulation partition refinement across all available units. This ensures GVMin to achieve a linear-time complexity of $O(|V^L| + |E^L|)$ given L and k are small constants, making it highly efficient and scalable for large-scale applications.

Forgetting Model. We also introduce a dynamic maintenance strategy, that aim to keep "useful" GMVs for the query stream as long as possible, hence maximize the reusability of GMVs and reduce the selection cost. Our strategy, inspired by established Ebbinghaus' Forgetting Curve theory [35, 39], poses a forgetting curve model to maintain important memory graphs for GNN M. Following the principle that memory decays exponentially over time, and most forgetting happens soon after learning or inference with a "rate of forgetting" slows down over time (what you remember after some time tends to remain stable longer), We use a view cache to swap out memory graphs that are no longer "fresh" and retain those expected to be needed ($\mathcal{V}_R$), following a forgetting curve expressed by an exponential decay model. Specifically, each view $\mathcal{V}$ is assigned a retention score r:

$$r(\mathcal{V}_i) = e^{-\frac{T_{ep}(\mathcal{V}_i)}{|S(\mathcal{V}_i)|}}$$

where e (the Euler's number) is a constant 2.71828, $S(\mathcal{V}_i)$ refers to the fraction of $N^L(v)$ covered by the memory graph G_i of $\mathcal{V}_i$, and $T_{ep}(\mathcal{V}_i)$ is the time elapsed since $\mathcal{V}_i$ is referred to for evaluating any inference query. In practice, the view cache is dynamically maintained by the procedure GVSel as top $m * k$ views with highest retention scores where $m \geq 1$. The ranking of retention scores ensure that the candidate views in view cache are always the most relevant views to answering Q among the set of all available views $\mathcal{V}$, so GVMin can potentially cover Q with fewer views. Meanwhile, GVMin can also be accelerated since its time complexity is reduced from $O(k|\mathcal{V}|)$ to $O(k|\mathcal{V}_R|)$.

Example 6: Continuing Example 4, two views $\mathcal{V}_2$ and $\mathcal{V}_4$ are selected at t_1. At t_1, $V_R = \{\mathcal{V}_2, \mathcal{V}_4, \mathcal{V}_3\}$. Assume $k = 2$ and cache size $m = 3$, and the time elapse between t_2 and t_1 is equal to 1. Upon receiving a new inference query Q' at t_2, forgetting model updates the retention scores for all the views. For the freshly received view $\mathcal{V}_1$, time elapsed $T_{ep}(\mathcal{V}_1) = 0$ and the size of the fraction covered $|S(\mathcal{V}_1)| = 9$, thus its retention score $r(\mathcal{V}_1) = e^{-\frac{0}{9}}$. For V_4, since $t_2 - t_1 = 1$, the time elapsed $T_{ep}(\mathcal{V}_4) = 5 + 1 = 6$ and $|S(\mathcal{V}_4)| = 10$, $r(\mathcal{V}_4) = e^{-\frac{6}{10}}$. For V_3, $T_{ep}(\mathcal{V}_3) = 10 + 1 = 11$ and $|S(\mathcal{V}_3)| = 9$. For

GNNs Classes	Node Update Function (general form)	Training Cost	Inference Cost
Vanilla [40]	$X_v^k = \sigma(\Theta \cdot \text{AGG}(X_u^{k-1}, \forall u \in \mathcal{N}(v)))$	$O(L\|E\| + L\|V\|)$	$O(L\|E\| + L\|V\|)$
GCN [11, 27]	$X_v^k = \sigma(\Theta^k(\sum_{u\in\mathcal{N}(v)} \frac{1}{\sqrt{d_u d_v}} x_u^{k-1}))$	$O(L\|E\|F + L\|V\|F^2)$	$O(L\|E\|F + L\|V\|F^2)$
GAT [7, 44]	$X_v^k = \sigma(\sum_{u\in\mathcal{N}(v)} \alpha_{uv}\Theta^k X_u^{k-1}))$	$O(L\|E\|dF^2 + L\|V\|F^2)$	$O(L\|E\|dF^2 + L\|V\|F^2)$
GraphSAGE [11, 20]	$X_v^k = \sigma(\Theta^k \cdot (X_v^{k-1} \| \text{AGG}(X_u^{k-1}, \forall u \in \mathcal{N}(v))))$	$O(L\|V\|dF + L\|V\|F^2)$	$O(L\|V\|dF + L\|V\|F^2)$
GIN [7, 49]	$X_v^k = \sigma(\text{MLP}((1+\gamma)x_v^{k-1} + \sum_{u\in\mathcal{N}(v)} x_u^{k-1})$	$O(L\|E\|F + L\|V\|F^2)$	$O(L\|E\|F + L\|V\|F^2)$

Table 6: Summary of Representative GNNs. σ: an activation function *e.g.,* ReLU or LeakyReLU. AGG: aggregation function *e.g.,* sum ($\sum$), average (Avg), or concatenation (||). L, $|E|$, $|V|$, F d: the number of layers, edges, nodes, features per node, and maximum degree.

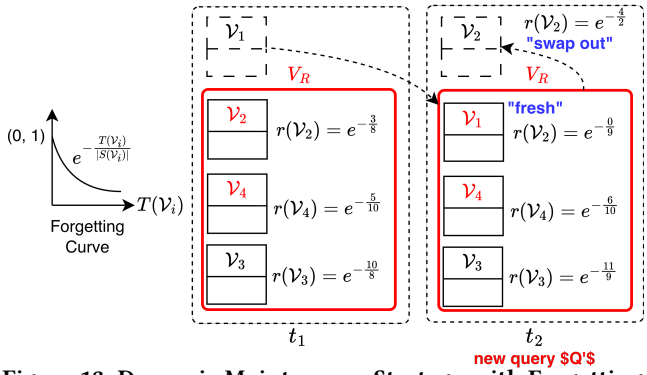


Figure 13: Dynamic Maintenance Strategy with Forgetting Mechanism. Discarding stale views in $\mathcal{V}$ and serves only the current "fresh" subset $\mathcal{V}_R$.

V_2, $T_{ep}(\mathcal{V}_2) = 3 + 1 = 4$ and $|S(\mathcal{V}_2)| = 2$. At t_2, $r(\mathcal{V}_2) = e^{-\frac{4}{2}}$ is smaller than $r(\mathcal{V}_1)$, $r(\mathcal{V}_4)$, and $r(\mathcal{V}_3)$, forgetting model swaps out the view $\mathcal{V}_2$ at t_2 for view $\mathcal{V}_1$. Therefore, at t_2, $V_R = \{\mathcal{V}_1, \mathcal{V}_4, \mathcal{V}_3\}$. Since $k = 2$, only two views $\mathcal{V}_1$ and $\mathcal{V}_4$ are selected at t_2. □

9.3 Appendix C: Complementary Experimental Study and Discussion

View Generation. In our experiments, graph model views (GMVs) are generated by mining previously observed graph and their stored inference outputs to identify recurring structural and patterns, akin to materialized view selection in traditional databases. We extract subgraphs that serve as candidate views, then select a compact set of such views by filtering the skewed cases to ensure that the coverage of selected views follows approximately a normal distribution. This approach adapts classical materialized-view design principles from the database literature [2, 50] to the graph inference setting. To ensure experimental balance, we generate 500 views per dataset, converting 50% into virtual views, discarding prior inference results and auxiliary structure, while retaining the remaining 50% as materialized views.

Exp-4: Ablation Analysis. Fixing $|W|$ being 500, GNN class being a 3-layers GraphSAGE, $|\mathcal{V}|$ being 500, and k being 10, we conduct ablation studies to evaluate how essential components selection GVSel, view minimization GVMin, and forgetting mechanism FM affects the inference efficiency (including inference time per query load and memory usage of V_m per test node) and inference accuracy (%). We present results computed on average across all datasets as illustrated in Table. 8.

W vs. W/O GVSel. Removing view selection leads to a substantial 14.17% accuracy drop (from 74.24% to 63.72%), 62.03% increases in memory usage per test node and increases inference time by 31.93% from 5.92 s to 7.81 s per query load. This verifies that GVSel effectively prioritizes informative views and reduces redundant information, leading to improved accuracy and efficiency.

W vs. W/O GVMin. Without GVMin, the inference accuracy drops slightly (from 74.24% to 73.21%) while leading to 135.64% increases in inference time from 5.92 s to 13.95 s per query load and 175.32% increases in memory usage per test node. This indicates that the effectiveness of GVMin serve as a computational optimization tool to accelerate inference and reduce the memory footprints without harming inference accuracy.

W vs. W/O FM. Disabling the forgetting mechanism, the inference accuracy remains nearly unchanged. However, the inference time rises up 7.09% from 5.92 to 6.34 s per query load and memory usage per test node increases by 36.08%. This demonstrates that the FM component primarily functions as a lightweight optimization strategy to accelerate inference by disregarding stale or less relevant cached views while preserving inference accuracy.

Exp-6: Memory Costs. We conduct memory cost analysis on ***Arxiv***, **Yelp**, and ***Ogbn-Products*** datasets. For each dataset, we report the memory costs of the original graph G and the generated 500 views, including the memory cost of all the memory graphs and all the auxiliary structure respectively. We define memory compression rate (mcr) as $\text{mcr} = 1 - \frac{|M_c|}{|M|}$. It quantifies the fraction of memory cost that is "reduced": the larger, the better; The detailed results are presented in Table 7. We conduct the following tests:

(1) **Graph Memory (GM) Test**: We measure the graph memory usage as the graph size in the PyTorch Geometric Graph Data format [16] including node features, node labels, and edge index tensors. Comparing to G, the GM for memory graphs much smaller, reduced by 75.30%, 84.32%, and 85.76% compared to the original graph G for ***Arxiv***, **Yelp**, and ***Ogbn-Products*** respectively;

(2) **Auxiliary Structure $\mathcal{T}$ Memory (ASM) Test**: We evaluate the memory cost of the auxiliary structure $\mathcal{T}$. We observe the followings: i): On average, the memory cost of $\mathcal{T}$ accounts for only 12.13% of that of G_c on average across all the three datasets; and ii) The combined memory cost of $G_c + \mathcal{T}$ still remains significantly smaller than that of G, with reductions of 72.52%, 83.19%, and 83.54% for ***Arxiv***, **Yelp**, and ***Ogbn-Products*** respectively;

(3) **Peak Memory (PM) Test**: We measure the peak memory usage during the inference process using a 3-layers GCN model, using a hidden size of 32. Compared to inference over the original graph G, inference over the set of views $\mathcal{V}$ reduces the peak memory by

Table 7: Memory cost comparison between the original graph G and the set of views graph $\mathcal{V}$.

Arxiv	$\|V\|+\|E\|$	GM	ASM	PM
G	1,335,586	101.13 MB	–	1.44 GB
$\mathcal{V}$	431,502	24.98 MB	2.81 MB	0.49 GB
mcr	–	75.30%	–	65.97%
Yelp	$\|V\|+\|E\|$	GM	ASM	PM
G	14,671,666	1036.03 MB	–	14.41 GB
$\mathcal{V}$	2,742,325	162.51 MB	11.56 MB	2.65 GB
mcr	–	84.32%	–	81.61%
Ogbn-Products	$\|V\|+\|E\|$	GM	ASM	PM
G	64,308,169	1887.47 MB	–	30.12 GB
$\mathcal{V}$	11,105,726	268.75 MB	41.78 MB	5.24 GB
mcr	–	85.76%	–	82.60%

Table 8: Ablation Study

Component	Accuracy (%)	Inf. ($s/\|Q\|$)	Mem. (MB/v)
GVInf	74.24%	5.92	1.58
w/o GVSel	63.72% (↓ 14.17%)	7.81 (↑ 31.93%)	2.56 (↑ 62.03%)
w/o GVMin	73.21% (↓ 1.39%)	13.95 (↑ 135.64%)	4.35 (↑ 175.32%)
w/o FM	74.03% (↓ 0.28%)	6.34 (↑ 7.09%)	2.15 (↑ 36.08%)

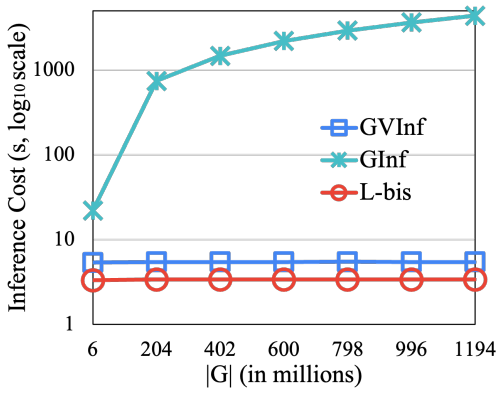


Figure 14: Time Costs of L-bisimulation versus. GVInf **and** GInf**.**

65.97%, 81.61%, and 82.60% for ***Arxiv***, **Yelp**, and ***Ogbn-Products***. Notably, as the size of original graph increases, the reduction in peak memory becomes more pronounced, reflecting the overall decrease in both graph size and memory footprints.

Exp-7: Time Costs of L-bisimulation. We report the time cost of computing L-bisimulation. Fixing $|Q|$ being 100, $|\mathcal{V}|$ being 500, k being 10, GNNs class being a 3-layers GAT, utilizing the simulated graph dataset **WS-MAG**, we vary the size of graph $|G|$ from 6 million to 1.194 billion to study how graph size affects the time cost of computing L-bisimulation, compared to inference time cost of GVInf and GInf. As $|G|$ increases from 6 million to 1.194 billion, the inference costs of computing L-simulation remains stable regardless of the graph size, as their costs are determined by the size of GMVs $|\mathcal{V}|$ while being significantly faster than GVInf and GInf.